BUKU BESAR
ELEKTRONIKA DIGITAL

KUMPULAN MATERI ELEKTRONIKA DIGITAL
Buku besar ini dibuat untuk memenuhi tugas mata kuliah Elektronika Digital

disusun oleh :

Meisya Angelivia
(221344016)

Septia Yauma Nur Falah
(221344027)

Nur Alim Lesmana
(221344025)

Silfiana Dwi Nabiilah
(221344028)



PROGRAM STUDI D4 TEKNIK TELEKOMUNKASI
JURUSAN TEKNIK ELEKTRO
POLITEKNIK NEGERI BANDUNG

2022

PRAKATA

Puji dan syukur senantiasa kami panjatkan kepada Tuhan Yang Maha Esa atas rahmat yang telah diberikan-Nya sehingga tugas “Buku Besar” ini dapat diselesaikan dengan tepat waktu dan dalam bentuk yang sebaik-baiknya. Buku Besar ini dibuat dengan tujuan untuk memenuhi tugas mata kuliah Elektronika Digital Dasar di Semester II ini.

Terima kasih kami ucapkan kepada Ferry Satria, BSEE., M.T. dan Rahmawati Hasanah, S.ST., M.T. selaku dosen mata kuliah Elektronika Digital Dasar karena telah memberikan tugas ini sebagai upaya kami dalam memperluas wawasan mengenai materi dasar pada Program Studi Teknik Telekomunikasi Nirkabel, lebih tepatnya dalam mata kuliah Elektronika Digital Dasar.

Dari pembuatan makalah ini, diakui bahwa masih adanya kekurangan. Oleh karena itu, saran dan kritik sangat dibutuhkan sebagai sarana perbaikan untuk tugas ini ataupun tugas selanjutnya. Semoga segala isi yang dimuat dapat bermanfaat bagi segala pihak yang membaca dan terlibat, serta tentunya bagi penulis sendiri.

Penulis,

Bandung, 07 Mei 2023

DAFTAR ISI

PRAKATA ii
DAFTAR ISI iii
DAFTAR GAMBAR vi
BAB 1 8
1.1 Pengertian Gerbang Logika 8
1.2 Jenis – Jenis Basic Gates 9
1.3 Pengertian Aljabar Boolean 13
1.4 Persamaan – Persamaan Aljabar Boolean 14
1.5 Sifat Dualitas 15
1.6 Pembuktian Aljabar Boolean Mengguankan Tabel Persamaan 17
1.7 Pertanyaan dan Jawaban Basic Gates 18
1.8 Menyederhanakan Persamaan 18
BAB 2 20
2.1 Pengertian Sistem Bilangan 20
2.2 Karnaugh Map 20
2.2.1 Langkah – langkah pemetaan K-Map secara umum: 22
2.2.2 Jenis-jenis K-Map: 22
2.3 Jenis – Jenis Sistem Bilangan 23
2.4 Mengubah Bilangan Biner Menjadi Bilangan Desimal 30
2.5 Mengubah Bilangan Heksadesimal Menjadi Bilangan Desimal 31
2.6 Mengubah Bilangan Biner Menjadi Bilangan Heksadesimal 32
2.7 Mengubah Bilangan BCD (EX-3) Menjadi Bilangan Desimal 32
2.8 Soal Jawab Sistem Bilangan 32

BAB 3 34
3.1 *Adder* (Penjumlahan).....34
3.1.1 Perbedaan Sum dan Carry.....34
3.1.2 Rangkaian Half Adder 35
3.1.3 Rangkaian Full Adder.....36
3.1.4 Rangkaian Parallel Adder 38
3.1.5 Rangkaian Carry Ahead Look Adder 41
3.1.6 BCD Adder 42
3.1.7 Rangkaian Penjumlahan 2 Data 4 Bit. 43
3.1.8 Contoh Aplikasi Rangkaian Decoder 44
3.1.9 Contoh Soal Mengenai Rangkaian Full Adder.....46
3.2 Substractor.....47
3.2.1 Half Subtractor.....47
3.2.2 Full Substractor.....48
3.3 Encoder.....49
3.3.1 Encoder 2 ke 1 50
3.3.2 Encoder 4 ke 2 51
3.3.3 Priority Encoder 4 ke 2 52
3.3.4 Konversi Oktal ke Biner dengan Encoder 8 to 3 53
3.3.5 Konversi Biner ke Oktal dengan Encoder 8 to 3 54
3.4 Multiplexer 55
3.4.1 MUX – 2 Kanal.....55
3.4.2 MUX 4 kanal 56
3.4.3 Aplikasi Rangkaian Multiplexer.....57

3.5 DeMultiplexer 59
3.5.1 deMUX – 2 Kanal 60
3.5.2 deMUX 4 kanal 60
3.6 Comparator 61
BAB 4 64
4.1 Pengertian Seven Segment Display 64
4.2 Jenis – Jenis Seven Segment Display 64
4.3 Prinsip Kerja Dasar 66
4.4 BCD to Seven Segment Display 66
BAB 5 75
5.1 Pengertian Parity Generator dan Parity Checker 75
5.2 Jenis – Jenis Parity Generator dan Parity Checker 76
BAB 6 83
6.1 Pegertian dan Fungsi Proteus 83
6.2 Cara Menginstal Proteus pada Window 84
6.3 Langkah Membuat Rangkaian pada Proteus 84
6.4 Contoh dan Latihan Soal 85
DAFTAR PUSTAKA 92

DAFTAR GAMBAR

Gambar 1. 1 Simbol AND Gate 9
Gambar 1. 2 Diagram Blok IC 7408 9
Gambar 1. 3 Simbol OR Gate 10
Gambar 1. 4 Diagram Blok IC 7432 10
Gambar 1. 5 Simbol NOT Gate 10
Gambar 1. 6 Diagram Blok IC 7404 10
Gambar 1. 7 Simbol NAND Gate 11
Gambar 1. 8 Diagram Blok IC 7400 11
Gambar 1. 9 Simbol NOR Gate 11
Gambar 1. 10 Diagram Blok IC 7402 12
Gambar 1. 11 Simbol EX-OR Gate.....12
Gambar 1. 12 Diagram Blok IC 7486 12
Gambar 1. 13 Simbol EX-NOR Gate.....13
Gambar 1. 14 Diagram Blok IC 4077 13
Gambar 2. 1 K-Maps 2 Variabel 22
Gambar 2. 2 K-Maps 3 Variabel 22
Gambar 2. 3 K-Maps 4 Variabel 22
Gambar 2. 4 K-Maps 5 Variabel 23
Gambar 3. 1 K-Map penyelesaian persamaan Y = A(B+C)+(B+C).....33
Gambar 3. 2 Rangkaian Half Adder.....35
Gambar 3. 3 Rangkaian Full Adder 36
Gambar 3. 4 Rangkaian Dari Dua Penjumlahan Bilangan Biner.....38
Gambar 3. 5 Rangkaian Dari Tiga Penjumlahan Bilangan Biner 39
Gambar 3. 6 Rangkaian Parallel Adder Dengan Waktu Kerja yang Dipersingkat 40
Gambar 3. 7 Simbol dan Rangkaian Carry Ahead Adder.....42
Gambar 3. 8 IC Rangkaian Parallel Binary Adder.....43
Gambar 3. 9 Rangkaian Penjumlahan Full Adder 46
Gambar 3. 10 Rangkaian Half Subtractor 47
Gambar 3. 11 Rangkaian Full Subtractor.....48

Gambar 3. 12 Simbol Encoder 4 ke 2 51
Gambar 3. 13 Rangkaian Encoder 4 ke 2.....52
Gambar 3. 14 K-Map Parity Encoder 4 to 2 53
Gambar 3. 15 Rangkaian Konversi ke Biner dengan Encoder 8 to 3 54
Gambar 3. 16 Simbol, Tabel Kebenaran dan K-maps Mux 2 Kanal 55
Gambar 3. 17 Rangkaian Mux 2 Kanal menggunakan IC 7408, 7404 dan 7432 ..56
Gambar 3. 18 Simbol dan Tabel Kebenaran 4 Kanal.....56
Gambar 3. 19 Rangkaian Mux 4 Kanal.....57
Gambar 3. 20 Rangkaian MUX Jika B Menjadi Selector.....57
Gambar 3. 21 Rangkaian MUX Jika A Menjadi Selector.....58
Gambar 3. 22 Rangkaian Mux 2 Kanal Jika Menggunakan K-Map.....59
Gambar 3. 23 Rangkaian DeMux 2 Kanal.....60
Gambar 3. 24 Rangkaian DeMux 4 Kanal.....61
Gambar 3. 25 Rangkaian Comparator 2 Bit.....62
Gambar 4. 1 Rangkaian Common Cathode 65
Gambar 4. 2 Rangkaian Common Anode 65
Gambar 4. 3 Blok Dasar Seven Segment Diplay 66
Gambar 4. 4 Format decoder tampilan 7 segment 68
Gambar 4. 5 IC BCD to Seven Segment Display 68
Gambar 4. 6 BCD to Seven Segment dengan Decoder Sederhana Pada Proteous 71
Gambar 5. 1 Rangkaian Parity Generator dan Parity Checker.....75
Gambar 5. 2 Rangkaian PoG.....77
Gambar 5. 3 Rangkaian PEC 79
Gambar 5. 4 Rangkaian PoG.....80
Gambar 5. 5 Rangkaian PEC 82

BAB 1
GERBANG LOGIKA DASAR DAN ALJABAR BOOLEAN

1.1 Pengertian Gerbang Logika

Elektronika dapat diartikan sebagai suatu sistem elektronik yang dapat dihubungkan dengan sebuah sinyal digital. Elektronika digital berasal dari aljabar Boolean dan digunakan pada banyak perangkat, seperti telepon, komputer, dan sebagainya. Pada dasarnya sinyal digital memiliki sifat terputus-putus. Selain itu, sinyal digital dapat dilambangkan dengan suatu notasi aljabar 1 dan 0. Notasi 1 melambangkan terjadinya sebuah sinyal digital pada suatu benda. Sedangkan notasi 0 melambangkan tidak adanya hubungan pada suatu benda. Elektronika digital dikenal dengan sebutan lain yaitu rangkaian digital. Penyusun dari rangkaian digital disebut dengan gerbang logika. Biasanya gerbang logika menjalankan operasi logika pada satu atau lebih masukan serta menghasilkan keluaran yang tunggal.

Gerbang logika memiliki beberapa bagian, yaitu resistor, transistor, dan dioda. Ketiga bagian tersebut akan melakukan operasi sederhana atau operasi kompleks dengan menggabungkan beberapa gerbang logika. Pada gerbang logika terdapat bagian yang bernama sirkuit terpadu (IC). IC memiliki fungsi untuk membatasi jumlah maksimum yang ada di gerbang logika. Sebelum adanya peningkatan pada sirkuit terpadu (IC), jumlah gerbang logika yang dibutuhkan pada elektronik digital sangat sedikit.

Setiap operasi gerbang logika akan direpresentasikan menggunakan tabel kebenaran. Dalam tabel kebenaran, angka “1” menandakan bahwa bernilai benar (true) dan angka “0” bernilai salah (false). Dengan sistem teknologi yang semakin berkembang, maka sistem elektronika akan semakin berkembang pula.

Basic Gates atau gerbang-gerbang dasar merupakan gerbang logika yang digunakan dalam rangkian elektronika. Gerbang tersebut memiliki

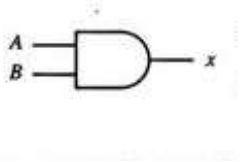
simbol masing-masing untuk setiap gerbangnnya. Ada tujuh gerbang dasar, yaitu AND Gate, OR Gate, NOT Gate, NAND Gate, NOR Gate, EX-OR Gate, dan EX-NOR Gate.

Basic Gates berkaitan dengan Boolean Algebra yang digunakan sebagai persamaaannya, serta mengetahui input dan outputnya dengan hanya memuat 0 (*no*/tidak ada) dan 1 (*yes*/ada). Hubungan antara input (masukan) dan output (keluaran) disajikan dalam sebuah tabel yang disebut dengan *truth table* atau tabel kebenaran.

1.2 Jenis – Jenis Basic Gates

A. AND Gate: Disebut juga dengan Gerbang DAN, memiliki dua masukan dan satu keluaran dengan IC 7408 sebagai contoh yang memuat Gerbang DAN.

Simbol	Persamaan	Tabel kebenaran



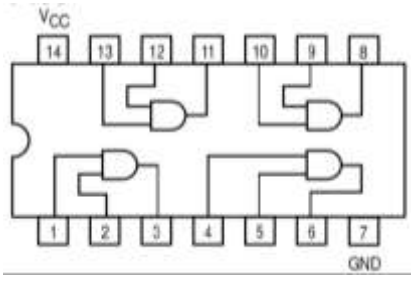
Gambar 1. 1 Simbol AND Gate

$$x = A.B = AB$$

X

input		output
A (1)	B (2)	X (3)
0	0	0
0	1	0
1	0	0
1	1	1

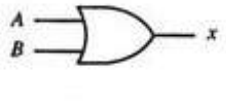
X



Gambar 1. 2 Diagram Blok IC 7408

B. OR Gate: Disebut juga dengan Gerbang OR, memiliki dua masukan dan satu keluaran dengan IC 7432 sebagai contoh yang memuat Gerbang OR.

Simbol	Persamaan	Tabel kebenaran



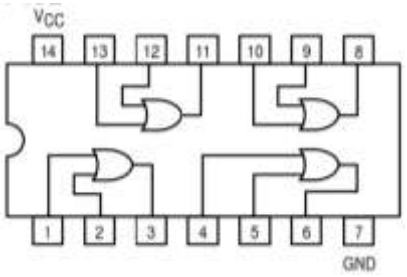
Gambar 1. 3 Simbol OR Gate

$$x = A + B$$

X

input		output
A (1)	B (2)	X (3)
0	0	0
0	1	1
1	0	1
1	1	1

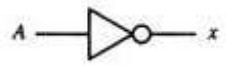
X



Gambar 1. 4 Diagram Blok IC 7432

C. NOT Gate: Disebut juga dengan Gerbang NOT atau inverter. Hanya memiliki satu masukan dan satu keluaran dengan fungsi sebagai pembalik. IC 7404 merupakan contoh yang memuat Gerbang NOT.

Simbol	Persamaan	Tabel kebenaran



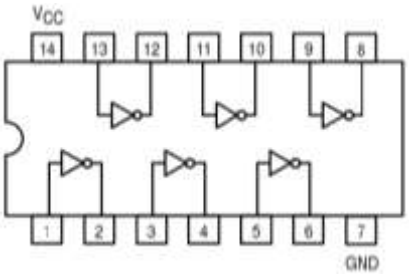
Gambar 1. 5 Simbol NOT Gate

$$x = \overline{A}$$

X

input	output
A (1)	X (2)
0	1
1	0

X

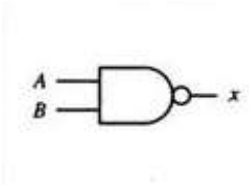


Gambar 1. 6 Diagram Blok IC 7404

D. NAND Gate: Merupakan gabungan dari NOT-AND atau kebalikan dari Gerbang AND jika dilihat dari outputnya. Gerbang NAND memiliki dua

masukan dan satu keluaran dengan IC 7400 sebagai contoh yang memuat Gerbang NAND.

Simbol



Gambar 1. 7 Simbol NAND Gate

Persamaan

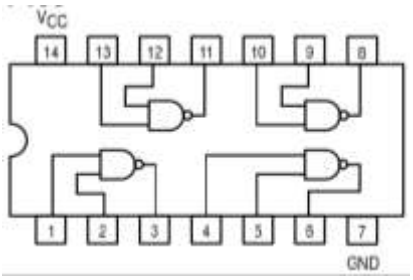
$$x = \overline{A.B} = \overline{AB}$$

Tabel kebenaran

X

input		output
A (1)	B (2)	X (3)
0	0	1
0	1	1
1	0	1
1	1	0

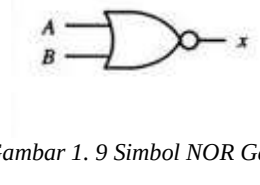
X



Gambar 1. 8 Diagram Blok IC 7400

E. NOR Gate: Merupakan gabungan dari NOT-OR atau kebalikan dari Gerbang OR jika dilihat dari outputnya. Gerbang NOR memiliki dua masukan dan satu keluaran dengan IC 7402 sebagai contoh yang memuat Gerbang NOR.

Simbol



Gambar 1. 9 Simbol NOR Gate

Persamaan

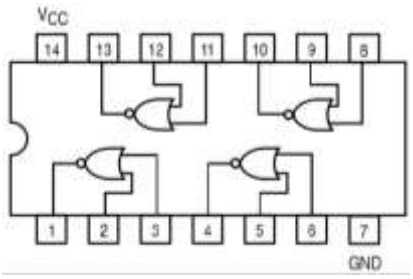
$$x = \overline{A + B}$$

Tabel kebenaran

X

input		output
A (3)	B (2)	X (1)
0	0	1
0	1	0
1	0	0
1	1	0

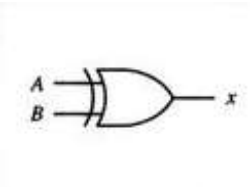
X



Gambar 1. 10 Diagram Blok IC 7402

F. EX-OR Gate: Yaitu Exclusive-OR, disebut juga dengan XOR Gate. Memiliki dua masukan dan satu keluaran dengan IC 7486 sebagai contoh yang memuat Gerbang XOR.

Simbol	Persamaan	Tabel kebenaran

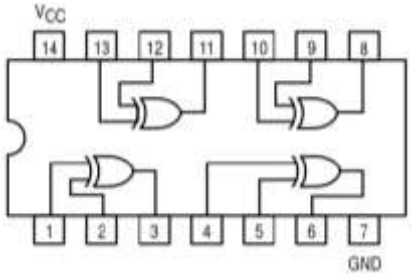


Gambar 1. 11 Simbol EX-OR Gate

$$x = A \oplus B = A\overline{B} + \overline{A}B$$

input		output
A (1)	B (2)	X (3)
0	0	0
0	1	1
1	0	1
1	1	0

X



Gambar 1. 12 Diagram Blok IC 7486

G. EX-NOR Gate: Memiliki dua masukan dan satu keluaran, dapat disebut juga dengan Exclusive NOR atau XNOR Gate. IC yang memuat Gerbang XNOR adalah IC 4077, bisa juga dengan menggabungkan antara IC 7486 dengan IC 7404.

Simbol	Persamaan	Tabel kebenaran



Gambar 1. 13 Simbol EX-NOR Gate

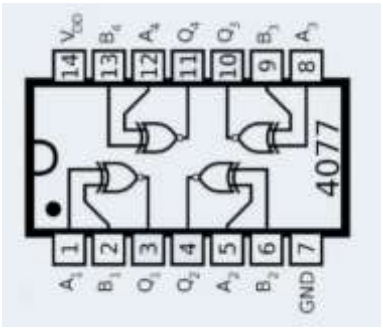
$$x = \overline{A \oplus B} = AB + \overline{A}\,\overline{B}$$

$$\overline{A}\,\overline{B} \neq \overline{AB}$$

X

input		output
A (1)	B (2)	X (3)
0	0	1
0	1	0
1	0	0
1	1	1

X



Gambar 1. 14 Diagram Blok IC 4077

1.3 Pengertian Aljabar Boolean

Boolean Algebra atau Aljabar Boolean merupakan aturan dasar logika yang membentuk struktur matematika. Aljabar Boolean ini ditemukan oleh George Boole pada tahun 1854 karena pada saat itu dia melihat bahwa himpunan dan logika porosisi memiliki sifat-sifat yang serupa.

Aljabar Boolean adalah operasi matematika yang berguna dalam menganalisis gerbang dan sirkuit digital, dengan menggunakan "Hukum Boolean" ini maka akan dapat mengurangi atau menyederhanakan ekspresi Boolean yang kompleks dengan maksud untuk mengurangi jumlah gerbang logika yang diperlukan.

Aljabar boolean adalah aljabar yang berhubungan dengan variabel biner dan operasi logik, dimana aljabar boolean adalah sistem matematika yang terbentuk dari 3 operator logika berupa "negasi", Logika "AND" dan

"OR". Selain simbol logika "0" dan "1" yang digunakan untuk merepresentasikan input atau output digital, kita juga dapat menggunakannya sebagai konstanta pada rangkaian terbuka atau rangkaian tertutup secara permanen.

1.4 Persamaan – Persamaan Aljabar Boolean

1. a) $0 \cdot 0 = 0$
   b) $1 + 1 = 1$

2. a) $1 \cdot 1 = 1$
   b) $0 + 0 = 0$

3. a) $0 \cdot 1 = 1 \cdot 0 = 0$
   b) $1 + 0 = 0 + 1 = 1$

4. a) Jika $x = 0$, maka $= 1$
   b) Jika $x = 1$, maka $= 0$

Variable Tunggal : x

5. a) $x \cdot 0 = 0$
   b) $x + 1 = 1$

6. a) $x \cdot 1 = x$
   b) $x + 0 = x$

7. a) $x \cdot x = x$
   b) $x + x = x$

8. a) $x \cdot \overline{x} = 0$
   b) $x + \overline{x} = 1$

9. a) $\overline{\overline{x}} = \overline{x}$

2 Variabel; 3 Variabel

10. commutative: a) $x \cdot y = y \cdot x$
    b) $x + y = y + x$

11. associative: a) $x \cdot (y \cdot z) = (x \cdot y) \cdot z$
    b) $x + (y + z) = (x \cdot y) + z$

12. distributive: a) $x \cdot (y + z) = (x \cdot y) + (x \cdot z)$
    b) $x + (y \cdot z) = (x + y) \cdot (x + z)$

13. absorptive: a) $x + (x \cdot y) = x$
    b) $x \cdot (x + y) = x$

14. combining: a) $(x \cdot y) + (x \cdot \overline{y}) = x$
    b) $(x + y) \cdot (x + \overline{y}) = x$

15. teorema De Morgan a) $\overline{x \cdot y} = \overline{x} + \overline{y}$
    b) $\overline{x + y} = \overline{x} \cdot \overline{y}$

1.5 Sifat Dualitas

Sifat dualitas, ini diperoleh dengan mengganti operator AND dengan ekuivalent operator OR, sedangkan operator OR dengan operator AND (bit 0 dengan 1, bit 1 dengan 0). Prinsip ini akan berguna dalam memanipulasi aljabar boole dalam penyederhanaan rangkaian logika.

Dualitas ekspresi Boolean diperoleh dengan mengganti operator AND dengan ekivalen operator OR. Dan operator OR dengan ekivalen operator AND, bit ‘0’ dengan ekivalen bit ‘1’ dan bit ‘1’ dengan bit ‘0’ Prinsip ini akan berguna dalam manipulasi aljabar Boolean dalam penyederhanaan rangkaian logika.

Aljabar Boolean berkaitan dengan Basic Gates karena suatu fungsi Boolean dapat dinyatakan dalam tabel kebenaran. Begitu pun sebaliknya, suatu tabel kebenaran dapat dinyatakan sebagai suatu fungsi Boolean.

Suatu tabel kebenaran bahkan dapat dinyatakan dalam dua fungsi Boolean yang ekuivalen. Fungsi-fungsi persamaan yang diperoleh dari

suatu tabel kebenaran disebut sebagai *canonical form (*bentuk kanonik). *Canonical form* adalah ekspresi Boolean yang dinyatakan sebagai penjumlahan dari satu atau lebih (*minterm*) atau perkalian dari satu atau lebih (*maxterm*). Ada dua macam bentuk kanonik, yaitu

1. Penjumlahan dari hasil kali atau *sum-of-product* (SOP); dan
2. Perkalian dari hasil jumlah atau *product-of-sum* (POS).

A. Sum Of Product (SOP)

Disebut juga disjunctive normal form, merupakan ekspansi bagian minterm tabelkebenaran. Terdapat tiga syarat minterm, yaitu :

1. Variabel lengkap
2. Nilai logic bernilai 1
3. Sifatnya di AND kan

Contoh :

Input			Output
A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

$F = \overline{A}BC + A\overline{BC} + A\overline{B}C + AB\overline{C} + ABC$

Tahap :

1. Melihat jumlah logic yang bernilai 1
2. Memastikan input yang bernilai 0 menggunakan tanda bar.
3. Akhir hasil bersifat minterm.

B. Product Of Sum (POS)

Merupakan ekspansi bagian maxterm tabel kebenaran. Terdapat tiga syarat maxterm,yaitu :

1. Variabel lengkap
2. Nilai logic bernilai 0
3. Sifatnya di OR kan

Contoh :

Input			Output
A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

$F = \overline{A}BC + A\overline{BC} + A\overline{B}C + AB\overline{C} + ABC$

Tahap :

4. Melihat jumlah logic yang bernilai 1
5. Memastikan input yang bernilai 0 menggunakan tanda bar.
6. Akhir hasil bersifat minterm.

1.6 Pembuktian Aljabar Boolean Mengguankan Tabel Persamaan

1. (X + Y)' = X' . Y'

Gerbang NOR equivalent dengan Gerbang AND uang inputnya dikomplement.

X	Y	$\overline{X}$	$\overline{Y}$	X + Y	$\overline{X + Y}$	$X.\overline{Y}\ \overline{}$
0	0	1	1	0	1	1
0	1	1	0	1	0	0
1	0	0	1	1	0	0
1	1	0	0	1	0	0

2. (X . Y)' = X' + Y'

Gerbang NAND equivalent dengan Gerbang OR uang inputnya dikomplement.

X	Y	$\overline{X}$	$\overline{Y}$	X . Y	$X.\overline{Y}$	$\overline{X} + \overline{Y}$
0	0	1	1	0	1	1
0	1	1	0	0	1	1
1	0	0	1	0	1	1
1	1	0	0	1	0	0

Teorema De Morgan dapat digunakan untuk mengkonversi pernyataan AND/ORmenjadi pernyataan OR/AND

Contoh :

Z = A'B'C + A'BC + AB'C + ABC'

Z' = (A + B + C') . (A + B' + C') . (A' + B + C') + (A' + B' + C)

X + X = X

T + T + T + T + T + ….. + T = T

X + XY = X

X (1 + Y) = X (1) = X

$X\overline{Y} + \overline{XY} = \overline{XY}$

1.7 Pertanyaan dan Jawaban Basic Gates

1) Kapan AND Gate berlogic 1?
   → Ketika seluruh inputnya berlogic 1.
2) Kapan OR Gate berlogic 1?
   → Ketika salah satu atau lebih dari inputnya berlogic 1.
3) Kapan NOT Gate berlogic 1?
   → Ketika inputnya berlogic 0.
4) Kapan NAND Gate berlogic 1?
   → Ketika salah satu atau lebih dari inputnya berlogic 0.
5) Kapan NOR Gate berlogic 1?
   → Ketika seluruh inputnya berlogic 0.
6) Kapan EX-OR Gate berlogic 1?
   → Ketika inputnya berbeda.
7) Kapan EX-NOR Gate berlogic 1?
   → Ketika inputnya sama.

1.8 Menyederhanakan Persamaan

Suatu persamaan aljabar Boolean dapat disederhanakan seperti pada contoh soal-soal berikut ini.

$$\boldsymbol{F = \overline{B}C + A\overline{B} + \overline{A}C}$$
$$F = \overline{B}C(1) + A\overline{B} + \overline{A}C$$
$$F = \overline{B}C\left(A + \overline{A}\right) + A\overline{B} + \overline{A}C$$
$$F = A\overline{B}C + \overline{A}\,\overline{B}C + A\overline{B} + \overline{A}C$$
$$F = A\overline{B}(C + 1) + \overline{A}C(B + 1)$$
$$F = A\overline{B} + \overline{A}C$$

$$\boldsymbol{F = \overline{B}C + \overline{AB} + AC}$$
$$F = \overline{B}C(1) + \overline{AB} + AC$$
$$F = \overline{B}C\left(A + \overline{A}\right) + \overline{AB} + AC$$
$$F = A\overline{B}C + \overline{A}\,\overline{B}C + \overline{A}\,\overline{B} + AC$$
$$F = A\overline{B}(C + 1) + AC(B + 1)$$
$$F = \overline{A}B(C + 1) + A\overline{C}(B + 1)$$
$$F = \overline{A}B + A\overline{C}$$

$$\boldsymbol{F = B\overline{C} + AB + \overline{A}C}$$
$$F = B\overline{C}(1) + AB + \overline{A}C$$
$$F = B\overline{C}\left(A + \overline{A}\right) + AB + \overline{A}C$$
$$F = AB\overline{C} + \overline{A}B\overline{C} + AB + \overline{A}C$$
$$F = AB(C + 1) + A\overline{C}(B + 1)$$
$$F = AB + \overline{A}C$$

$$\boldsymbol{F = AB\overline{C} + AB\overline{D} + \overline{C}D}$$
$$F = AB\overline{C}(1) + AB\overline{D} + \overline{C}D$$
$$F = AB\overline{C}\left(D + \overline{D}\right) + AB\overline{D} + \overline{C}D$$

$$F = A\overline{B} + AC$$

$$\boldsymbol{F = BC + \overline{A}C + AB}$$

$$F = BC(1) + \overline{A}C + AB$$

$$F = BC\left(A + \overline{A}\right) + \overline{A}C + AB$$

$$F = ABC + \overline{A}BC + \overline{A}C + AB$$

$$F = AB(C + 1) + \overline{A}C(B + 1)$$

$$F = \overline{A}C + AB$$

$$\boldsymbol{F = B\overline{C} + \overline{A}B + A\overline{C}}$$

$$F = B\overline{C}(1) + \overline{A}B + A\overline{C}$$

$$F = B\overline{C}\left(A + \overline{A}\right) + \overline{A}B + A\overline{C}$$

$$F = AB\overline{C} + \overline{A}B\overline{C} + \overline{A}B + A\overline{C}$$

$$F = AB\overline{C}D + AB\overline{CD} + AB\overline{D} + \overline{C}D$$

$$F = AB\overline{D}(C + 1) + \overline{C}D(AB + 1)$$

$$F = AB\overline{D} + \overline{C}D$$

$$\boldsymbol{F = A\overline{B}C\overline{D} + A\overline{BE} + C\overline{D}E}$$

$$F = A\overline{B}C\overline{D}(1) + A\overline{BE} + C\overline{D}E$$

$$F = A\overline{B}C\overline{D}\left(E + \overline{E}\right) + A\overline{BE} + C\overline{D}E$$

$$F = A\overline{B}C\overline{D}E + A\overline{B}C\overline{DE} + A\overline{BE} + C\overline{D}E$$

$$F = A\overline{BE}(CD + 1) + C\overline{D}E(AB + 1)$$

$$F = A\overline{BE} + C\overline{D}E$$

BAB 2
SISTEM BILANGAN

2.1 Pengertian Sistem Bilangan

Sistem bilangan yang dimaksud adalah sistem bilangan yang digunakan pada sistem digital, ini merupakan suatu cara untuk mewakili besaran dari suatu item fisik.

2.2 Karnaugh Map

Ada beberapa cara yang dapat digunakan untuk melakukan penyedehanaan terhadap sebuah persamaan logika, salah satunya ialah Karnough Map. Karnough Map atau yang biasa disingkat K-Map adalah metode penyederhanaan dari sebuah persamaan aljabar Boolean (fungsi logika) dengan menggunakan cara pemetaan. K-Map terdiri dari kotak-kotak yang jumlahnya terdiri sebanyak jumlah variabel dari suatu persamaan atau sejumlah input (masukan) dari rangkaian logika yang sedang dihitung. Jumlah kotak dalam K-Map dapat dihitung menggunskan rumus 2^n, dengan *n* adalah banyak variabel.

Input (masukan) memiliki urutan yang berbeda dari tabel persamaan fungsi logika pada umumnya, yaitu 00, 01, 11, 10. Jika diperhatikan, urutan nilai 10 dan 11 bertukar, hal tersebut dikarenakan nilai-nilai yang ada telah disusun sedemikian rupa supaya pasangan setiap sel yang bersebelahan hanya memiliki perbedaan satu nilai saja.

Sebelum dilakukan proses minimisasi (penyederhanaan), perlu dilakukan pemetaan nilai-nilai output (keluaran) pada masing-masing selnya terlebih dahulu, dengan catatan bahwa tidak boleh ada sel yang kosong. Tiap sel harus diisi dengan nilai 0, 1, atau 0/X (don’t care).

Proses penyederhanaan dilakukan dengan melakukan pengelompokan setiap sel yang bersebelahan atau bertetangga penggabungan harus membentuk segi empat ,jumlah anngota pada suatu kelompok hanya boleh berjumlah 1,2,4,8,16 dst. Perlu diketahui bahwa

yang dimaksud dengan “setiap sel yang bersebelahan” seperti yang sebelumnya disebut, berlaku bagi seluruh sel dalam K-Map, baik vertikal maupun horizontal. Kolom pertama bersebelahan dengan kolom terakhir, baris paling atas pun bersebelahan dengan baris paling bawah.

Kemudian penyederhanaannya diambil dari gabungan sesedikit mungkin dengan anggota sel sebanyak mungkin. Jika suatu kelompok bisa dibuat dari gabungan 8 sel, maka tidak boleh digabungkan hanya dengan 4 sel saja. Sel atau gabungan sel yang sudah tergabung juga tetap dapat digabungkan berkali-kali sesuai dengan persamaan dari teorema $X + X + X + \cdots = X$, yang artinya suatu nilai akan tetap bernilai sama walaupun dijumlahkan berkali-kali. Persamaan untuk penyederhanaannya dilihat dari nilai yang konsisten.

cara menyederhanakan langsung ke paling sederhana tanpa bisa disederhanakan lagi. K-Map hanyalah salah satunya saja yang cukup terkenal karena dirasa yang paling cepat dan mudah.

Contoh:

Pada masukan $F = 0100 + 1100 + 0110 + 1110$ maka penyelesaiannya sebagai berikut.

F: AB \ CD	00	01	11	10
00	0	0	0	0
01	1	0	0	1
11	1	0	0	1
10	0	0	0	0

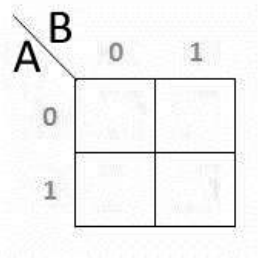
$$F = B\overline{D}$$

2.2.1 Langkah – langkah pemetaan K-Map secara umum:

1. Menyusun aljabar Boolean terlebih dahulu
2. Menggambar rangkaian digital
3. Membuat tabel kebenarannya
4. Merumuskan tabel kebenarannya
5. Lalu lakukan pemetaan dari tabel kebenaran ke dalam K-Map

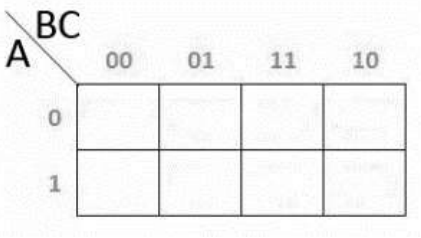
2.2.2 Jenis-jenis K-Map:

1. 2 Variabel



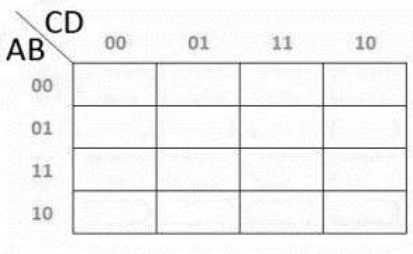
Gambar 2. 1 K-Maps 2 Variabel

2. 3 Variabel



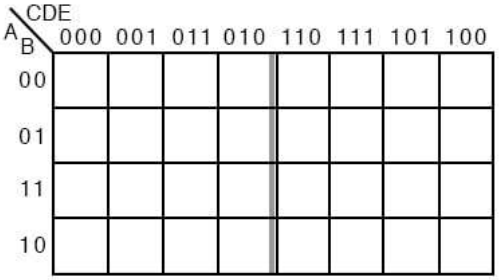
Gambar 2. 2 K-Maps 3 Variabel

3. 4 Variabel



Gambar 2. 3 K-Maps 4 Variabel

4. 5 Variabel



Gambar 2. 4 K-Maps 5 Variabel

2.3 Jenis – Jenis Sistem Bilangan

1) Bilangan Desimal

Desimal merupakan sistem bilangan dengan basis 10, artinya angka/digit yang digunakan untuk menyajikannya berjumlah 10 serta setiap digit penyusunannya memiliki bobot kepangkatan 10n dengan *n* merupakan bilangan bulat positif dan negatif. Nilai suatu sistem bilangan desimal memiliki karakteristik dimana besarnya nilai bilangan tersebut ditentukan oleh posisi atau tempat bilangan tersebut berada. Bilangan desimal adalah bilangan yang paling umum digunakan dalam kehidupan sehari-hari: yaitu 0, 1, 2, 3, 4, 5, 6, 7, 8, dan 9. Bilangan desimal dibagi menjadi tiga sebagai berikut :

1. Desimal murni, yaitu desimal yang disajikan dari angka 1-10
2. Komplemen 9 (K’9), yaitu untuk menyatakan bilangan bertanda positif (+), negative (-) dan nol (0). Komplemen 9 dari suatu bilangan desimal delakukan dengan cara mengurangkan angka 9 untuk masing – masing digit dalam bilangan pengurang.

   Contoh :

   1. Komplemen 9 dari 24 adalah 75, yaitu : 99 – 24 = 75
   2. Komplemen 9 dari 321 adalah 678, yaitu : 999 – 321 = 678

3. Komplemen 10 (K’10), yaitu nilai komplemen 9 ditambah 1 (K’9+1)

Contoh :

1. Komplemen 10 dari 123 adalah 124, yaitu : 123 + 1 = 124
2. Komplemen 10 dari 715 adalah 716, yaitu : 715 + 1 = 716

Note :

1. Jika bilangan bertanda (+) maka K'9 dan K'10 bernilai sama.
2. Jika bilangan bertanda (-) maka K'9 dan K'10 bernilai berbeda.
3. K'9 hanya berlaku jika billangan desimalnya adalah (-).

Contoh :

1. Desimal dari -123 jika di k'9 kan yaitu 876 dan k'10 876 + 1 = 877

Berikut ini adalah tabel komplemen 9 :

Des	K'9
0	9
1	8
2	7
3	6
4	5
5	4
6	3
7	2
8	1
9	0

2) Bilangan Biner

Biner atau basis 2 adalah sistem bilangan yang terdiri dari 2 simbol saja, yaitu 0 dan 1, kadang-kadang disebut juga sistem berbasis dua. Sistem bilangan ini merupakan dasar dari semua sistem bilangan berbasis digital. Dari sistem biner, kita dapat mengkonversinya ke sistem bilangan Oktal atau Hexadesimal. Sistem ini juga dapat kita sebut dengan istilah *bit*, atau *Binary Digit*. Pengelompokan biner dalam computer selalu berjumlah 8,

dengan istilah 1 Byte/bita. Bilangan biner dinyatakan dalam radik 2 atau disebut juga dengan sistem bilangan basis 2, dimana setiap *biner* atau *biner digit* disebut *bit.* Bilangan biner dibagi menjadi tiga sebagai berikut.

1. Biner murni, yaitu biasa disebut dengan bilangan Sign and Magnitude. Sign-and-Magnitude (SM) menyimpan nilai positif dan negatif dengan membagi bit total "n" menjadi dua bagian karena hanya memiliki dua simbol, yaitu 0 dan 1. Bilangan biner bernilai positif (+) tandanya akan digantikan oleh bit 0, sedangkan bilangan biner yang bernilai negatif (-) tandanya akan digantikan oleh bit 1.
2. Komplemen 1 (K'1), yaitu Komplemen 1 dari suatu bilangan biner dilakukan dengan cara mengurangkan semua digit dengan nilai bit 1 atau merubah bit '0' menjadi '1' atau bit '1' menjadi '0'. Dalam komplemen 1, digit 1 paling ujung kiri ditambahkan pada digit paling kanan.

   Note :

   1. Jika bilangan bernilai (+) maka bersifat seperti bilangan Sign Magnitude.
   2. Jika bilangan bernilai (-) maka bersifat berbanding terbalik dengan bilangan Sign Magnitude, dimana ketika bilangan bernilai 1 maka akan dibaca sebagai bilangan bernilai 0 begitupun sebaliknya.

   Contoh :

   1. Nilai K'1 dari 1111011 adalah 1111011
   2. Nilai K'1 dari -1111011 adalah -0000100

3. Komplemen 2 (K'2), yaitu nilai komplemen 1 ditambah 1 (K'1 + 1)

   Contoh :

   1. Komplemen 2 dari -1111011 adalah -0000101, yaitu : -1111011 + 1 = -0000101

   Berikut ini adalah tabel komplemen 1.

Des	K'1
0	1
1	0

3) Bilangan Oktal

Pada sistem computer jaman dulu, informasi biner dinyatakan dengan menggunakan bilangan octal. Sistem bilangan octal menggunakan delapan symbol yaitu 0,1, 2, 3, 4, 5, 6, dan 7. Bilangan octal juga berhubungan dengan bilangan dasar 8, menunjukkan keekuivalenan antara bilangan biner dan bilangan octal untuk bilangan desimal 0 sampai 17. Kelebihan dari bilangan octal adalah kegunaannya pada pengubah langsung dari sebuah bilangan kebilangan biner 3-bit. Notasi octal digunakan untuk menyatakan bilangan biner.

Cara penyalesaian bilangan oktal dapat dilakukan dengan 2 cara, yaitu :

1. Mengelompokkan masing-masing menajdi 3 bit
   Contoh :
   1. Nilai bilangan oktal dari 01111011 adalah 01111011 = 173
   2. Nilai bilangan oktal dari 1011001011 adalah 1011001011 = 1313
2. Membagi nilai biner dengan angka 8

Bilangan oktal dibagi menjadi tiga sebagai berikut :

1. Oktal murni
2. Komplemen 7 (K'7), yaitu Komplemen 7 dari suatu bilangan oktal dilakukan dengan cara, mengurangkan angka 7 untuk masing – masing digit dalam bilangan pengurangan.
   Contoh:
   1. Nilai K'7 dari -123 adalah -654
   2. Nilai K'7 dari -173 adalah -604
   Note : Nilai K'7 ini hanya berlaku pada kondisi bilangan negative, sebab ketika bilangan positif maka nilai K'7 akan sama dengan bilangan oktalnya.
3. Komplemen 8 (K'8), yaitu nilai komplemen 7 ditambah 1 (K'7 + 1)
   Contoh :
   1. Nilai K'8 dari -654 adalah -655

2. Nilai K’8 dari -604 adalah -605

Berikut ini adalah tabel komplemen 7 :

Des	K’7
0	7
1	6
2	5
3	4
4	3
5	2
6	1
7	0

4) Bilangan Heksadesimal

Sama seperti sistem bilangan lain yang memiliki jumlah sesuai dengan namanya, bilangan heksadesimal atau basis 16 berasal dari kata heksa yang berarti enam dan desimal yang memiliki arti sepuluh. Heksadesimal terdiri dari 16 simbol yaitu 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A (10), B (11), C (12), D (13), E (14), F (15). Penggunaan simbol yang dicampur dengan alfabet dikarenakan jumlah bilangan desimal yang hanya terdiri dari 10 angka, namun masih diperlukan simbol tambahan, maka digunakanlah simbol dari alfabet untuk mempermudah (sudah familiar dengan simbol pada alfabet). Sistem bilangan ini digunakan untuk menampilkan nilai alamat memori dalam pemrograman komputer.

Bilangan Heksadesimal dibagi menjadi tiga sebagai berikut :

1. Heksadesimal murni
2. Komplemen 15 (K’15), yaitu Komplemen 15 dari suatu bilangan hexadesimal dilakukan dengan cara, mengurangkan angka 15 untuk masing – masing digit dalam bilangan pengurangan.

   Contoh:

   1. Nilai K’15 dari -7B adalah -84

2. Nilai K’15 dari -2CB adalah -1334

Note : Nilai K’15 ini hanya berlaku pada kondisi bilangan negative, sebab ketika bilangan positif maka nilai K’15 akan sama dengan bilangan Heksadesimal nya.

3. Komplemen 16 (K’16), yaitu nilai komplemen 15 ditambah 1 (K’15 + 1)

Contoh :

1. Nilai K’16 dari -84 adalah -85
2. Nilai K’16 dari -1334 adalah -1335

Beikut ini adalah tabel komplemen 15 :

Des	K’15
0	F
1	E
2	D
3	C
4	B
5	A
6	9
7	8
8	7
9	6
A	5
B	4
C	3
D	2
E	1
F	0

5) Bilangan BCD

Binary Code Desimal atau Kode Biner Desimal adalah salah satu proses mengubah bilangan desimal menjadi ekivalen lainnya. Dalam sistem penomoran BCD, angka desimal dipisahkan menjadi empat bit untuk setiap digit desimal dalam angka tersebut. Berikut ini contoh jenis BCD.

BCD 8421

Setiap bilangan Desimal diwakili oleh bilangan biner 4 bit. Jadi dapat dikatakan bahwa Kode BCD (Binary Coded Decimal) adalah sistem pengkodean Biner dari angka Desimal dimana setiap digit Desimal diwakili oleh sejumlah bit, biasanya terdiri dari 4 bit. Asal usul disebut BCD 8421 diambil karena nilai 2n. Kode biner desimal ini adalah yang paling umum digunakan.

Contoh :

1. Nilai BCD 8421 dari 123 adalah 000100100011

2. Nilai BCD 8421 dari 635 adalah 011000110101

BCD (EX-3)

` Hampir sama dengan BCD 8421 hanya saja pada BCD (EX-3) ini nilai sebelum dijadikan menjadi biner bit 4 perlu ditambah 3 terlebih dahulu. Excess 3 artinya setiap bilangan dalam bentuk biner akan memiliki selisih tiga.

Contoh :

1. Nilai BCD 8421 (EX-3) dari 123 adalah 123 + 3 setiap bilangan = 456, maka nilai BCD (EX-3) nya 456 adalah 010001010110

2. Nilai BCD 8421 (EX-3) dari 635 adalah 635 + 3 setiap bilangan = 968, maka nilai BCD (EX-3) nya 968 adalah 100101101000

2.4 Mengubah Bilangan Biner Menjadi Bilangan Desimal

Terdapat 2 cara yaitu :

1) Cara 1

- Setiap biner memiliki koma biner
- Setiap desimal memiliki koma desimal

0	1	0	1	0	1	0	1	0	1	,	1	0	1	0	1	0	1	0	1
2^9	2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0		2^{-1}	2^{-2}	2^{-3}	2^{-4}	2^{-5}	2^{-6}	2^{-7}	2^{-8}	2^{-9}

Harga nilai masing-masing yaitu 0,5 dibagi 2 :

$2^{-1} = 0,5$

$2^{-2} = 0,25$

$2^{-3} = 0,125$

$2^{-4} = 0,0625$

$2^{-5} = 0,03125$

$2^{-6} = 0,015625$

$2^{-7} = 0,00078125$

$2^{-8} = 0,00390025$

$2^{-9} = 0,001953125$

Langkah :

1. Membuat tabel panjang untuk mengetahui letak bilangan yang akan dijumlahkan.
2. Menjumlahkan dengan nilai masing-masing

Contoh soal :

Biner 0,101010101

Maka : 0,5 + 0,125 + 0,03125 + 0,00078125 + 0,001953125 = 0,666015625

2) Cara 2

Langkah :

1. Hitung jumlah digit dibelakang koma.
2. Bagi hasil digit dengan 2n (n = jumlah bilangan dibelakang koma)
3. Hitung 4 digit pertama lalu kalikan dengan 2n (n = nilai digit sisanya)
4. Jumlahkan semuanya

Contoh soal :

1. Nilai desimal dari nilai biner 0,101010101

0,101010101 = jumlah digit dibelakang koma ada 9, maka $2^9 = 512$

Jumlah 4 digit pertama adalah 10

Jumlah digit sisa = $2^5 = 32$

Maka, 10 x 32 = 320

Sisa digit jika dalam biner adalah 21

Maka hasil akhir, $\frac{320+21}{512} = \frac{341}{512} = 0{,}666015625$

2. Nilai desimal dari nilai biner 101,1011101

101,1011101 = jumlah digit dibelakang koma ada 7, maka $2^7 = 128$

Jumlah 4 digit pertama adalah 11

Jumlah digit sisa = $2^3 = 8$

Maka, 11 x 8 = 88

Sisa digit jika dalam biner adalah 5

Maka hasil akhir, $\frac{88+5}{128} = \frac{93}{128} = 0{,}7265625$

Tiga digit di depan koma adalah 5

Maka hasil akhirnya, 5,7265625

2.5 Mengubah Bilangan Heksadesimal Menjadi Bilangan Desimal

Dengan mengalikan setiap sukunya untuk bentuk heksadesimal yang non-koma. Sedangkan untuk yang berkoma, dapat dikonversi dengan menjumlahkan setiap bilangannya dengan tabel koma.

2.6 Mengubah Bilangan Biner Menjadi Bilangan Heksadesimal

Dengan mengelompokkan ke dalam 4 bit pada bentuk biner, kemudian hitung nilai setiap kelompoknya. Jika dalam bentuk biner terdapat koma, maka pengelompokkan terbagi dua. Biner di belakang koma dikelompokkan ke arah kanan, sedangkan biner di depan koma dikelompokkan ke arah kiri. Contohnya sebagai berikut.

Biner : 10011.11101100

Heksadesimal : 0001 0011 . 1110 1100

(3) (14) (10)

= 13EC

2.7 Mengubah Bilangan BCD (EX-3) Menjadi Bilangan Desimal

Serupa dengan cara konversi dari bentuk biner ke heksadesimal, nilai BCD [EXCESS 3] dikelompokkan menjadi 4 bit terlebih dahulu. Contohnya sebagai berikut.

BCD [EX-3] : - 1010 1100 1001 1010

(12) (9) (10)

Desimal : - 7967

2.8 Soal Jawab Sistem Bilangan

1. Konversi Heksadesimal menuju decimal dari persamaan Y = 8AE

   Jawab :

   $8AE_{(16)} = ((8 \times 16) + 10) \times 16) + 14)) = 2222_{(10)}$

   $100010101110_{(16)} = 2222_{(10)}$

2. Konversi Biner menuju Heksadesimal dari persamaan 10011,111011

   Jawab :

   $10011{,}111011 = 13, EC$

3. Konversi BCD EX-3 menuju desimal dari persmaan -1010110010011010

   Jawab :

   $-1010110010011010 = -7967_{(10)}$

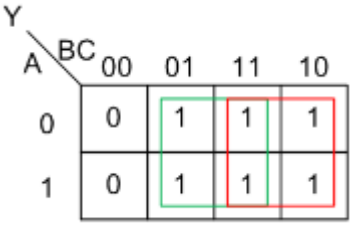
4. Penyelesaian menggunakan K-Maps dari persamaan $Y = A(B + C) + (B + C)$

Jawab :

$Y = A(B + C) + (B + C)$

$= (A + 1) + (B + C)$

$Y = B + C$



Gambar 3. 1 K-Map penyelesaian persamaan Y = A(B+C)+(B+C)

BAB 3
LOGIKA KOMBINASI DALAM KEMASAN IC

3.1 *Adder* (Penjumlahan)

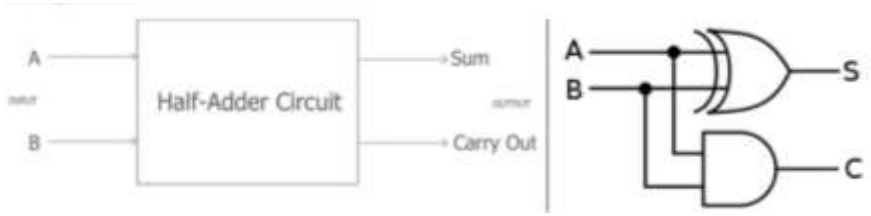
Rangkaian adder/penjumlah biasanya berada dalam processor, tepatnya dalam *ALU (Arithmetic Logic Unit).* Ada tiga jenis rangkaian ini yakni half adder, full adder dan ripple carry adder. Seperti kita tahu bahwa processor menggunakan *basic binary* untuk melakukan penghitungan sebuah proses, ada proses penghitungan aritmatik (menambah, mengurang, mengali dan membagi) dan ada pula proses menghitung logic (and, or, not, dst).

Adder digunakan untuk melakukan penghitungan aritmatik, terutama penjumlahan, pada prinsipnya processor akan memasukan 2 buah input untuk dijumlah sehingga didapatkan hasil SUM (S) dan CARRY (C). Sum adalah hasil penjumlahan pada position yang sama sedangkan Carry adalah kelebihan dari hasil penjumlahan yang melimpah pada posisi berikutnya. Pengujian *Adder* tersebut dilakukan untuk kedua jenis *adder*, yaitu *half adder* dan *full adder*. *Half adder* adalah rangkaian logika kombinasional yang dirancang dengan menghubungkan satu gerbang EX-OR dan satu gerbang AND, dengan memiliki dua input dan dua output (*carry out* dan *sum*). Output *sum* diperoleh dari gerbang EX-OR, sedangkan output *carry out* atau Co diperoleh dari gerbang AND. Berikut ini simbol dan tabel kebenarannya

3.1.1 Perbedaan Sum dan Carry

Misal perhitungan 5 ditambah 7. Kita tahu bahwa 5+7 = 12, tapi perhatikan lebih detail, baik 5 dan 7 keduanya nilai posisinya sama, yaitu satuan, penjumlahan keduanya menghasilkan bilangan Sum = 2 (satuan) dan karena nilai satuan berakhir pada angka 9 maka *nilainya melimpah* (*overflow*) pada posisi berikutnya (puluhan) sehingga muncul angka 1 (puluhan) yang disebut Carry. Dengan demikian 5+7 menghasilkan angka 12 { 1 (puluhan – Carry) 2 (satuan – Sum).

3.1.2 Rangkaian Half Adder



Gambar 3. 2 Rangkaian Half Adder

Rangkaian Half Adder adalah rangkaian yang memiliki dua buah input dan dua buah output, Sum bertindak sebagai output utama, sementara Carry Out berfungsi menerima limpahan nilai saat kedua nilai input berlogika 1.

Sirkuit Half adder terbentuk dari susunan gerbang logika, EXOR (7486) dan AND (7408). Nilai S pada rangkaian diartikan sebagai S = Sum, sedangkan C diartikan sebagai C = Carry Out.

Tabel Kebenaran Half Adder

input		output	
A	B	Co	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Jumlah input dari rangkaian half adder adalah 2, yakni input A dan B. Rangkaian Half Adder memiliki 2 buah output yaitu Carry dan Sum

Persamaan Half Adder :

$S = \overline{A}B + A\overline{B} = A \oplus B$

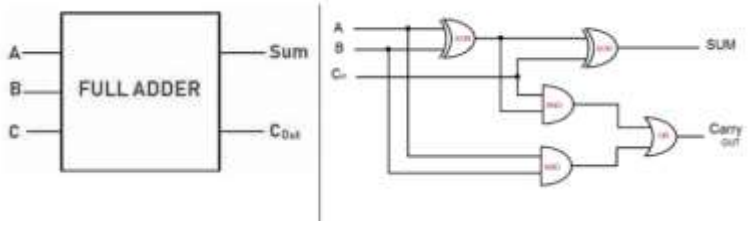
$Co = AB$

Pada saat A dan B = 1 maka Sum adalah 0 dan Carry menjadi 1.

Kekurangan dari rangkaian Half Adder adalah rangkaian digital tersebut hanya valid bertindak sebagai penghitung pertama dalam sebuah rangkaian penghitungan, maksudnya, jika kita melakukan 2 x operasi penjumlahan atau lebih, maka hasil dari rangkaian Half Adder tidak bisa dipastikan kebenarannya.

Misal kita telah menghasilkan angka 12 dari penjumlahan 5+7 di atas, kemudian pada saat penjumlahan berikutnya kita tambahkan dengan 9, jika kita menggunakan rangkaian half adder, maka hasil penjumlahannya adalah 2 (sebagai Sum penjumlahan pertama) ditambah 9, hasilnya adalah Carry 1 dan Sum 1 atau kita baca 11, padahal kita tahu hasil yang benar adalah 21. Kekurangan ini terjadi karena Half Adder hanya memiliki 2 input untuk dijumlahkan, yaitu A dan B.

3.1.3 Rangkaian Full Adder



Gambar 3. 3 Rangkaian Full Adder

Full Adder menyempurnakan kekurangan Half Adder dengan menambahkan 1 input lagi yaitu Carry In. Dengan demikian Full adder adalah suatu rangkaian penjumlahan yang mempunyai increment input. Rangkaian penjumlahan yang mempunyai increment input ini adalah Full Adder dan Ripple Carry Adder. Perbedaan full adder dibanding half adder adalah adanya sebuah input tambahan yakni Carry in. Dengan menambahkan carry in ini rangkaian full adder dapat memberikan output nilai yang tepat seandainya ada limpahan nilai carry dari penjumlahan sebelumnya.

Sirkuit Full adder terbentuk dari susunan gerbang logika, EXOR (7486), AND (7408) dan OR (7432). Pada input terdiri atas gerbang A, B, dan Cin. Sedangkan pada gerbang output terdiri atas gerbang Sum dan Carry out.

Tabel Kebenaran Full Adder

input			output	
A	B	Ci	Co	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Jumlah input dari rangkaian Full Adder adalah 3, yakni input A, B dan Cin. Rangkaian Full Adder memiliki 2 buah output yaitu Carry dan Sum

Persamaan Full Adder:

$$S = \overline{A\overline{B}}Cin + \overline{A}B\overline{Cin} + A\overline{B\overline{Cin}} + ABCin$$

$$S = \overline{A}(\overline{B}Cin + B\overline{Cin}) + A(\overline{B\overline{Cin}} + BCin)$$

$$S = \overline{A}(B \oplus Cin) + A\overline{(B \oplus Cin)}$$

$$S = A \oplus B \oplus Cin$$

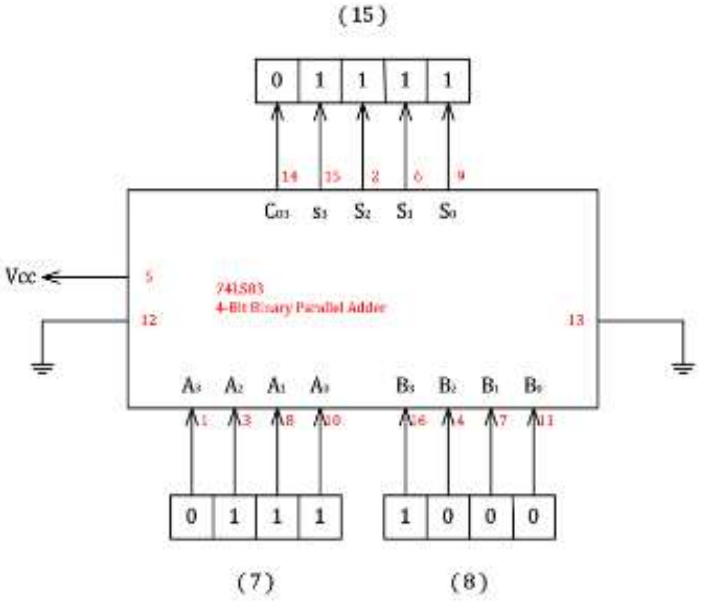
$$Co = ACin + AB + BCin$$

Pada saat A dan B = 1 maka Sum adalah 0 dan Carry menjadi1.

3.1.4 Rangkaian Parallel Adder

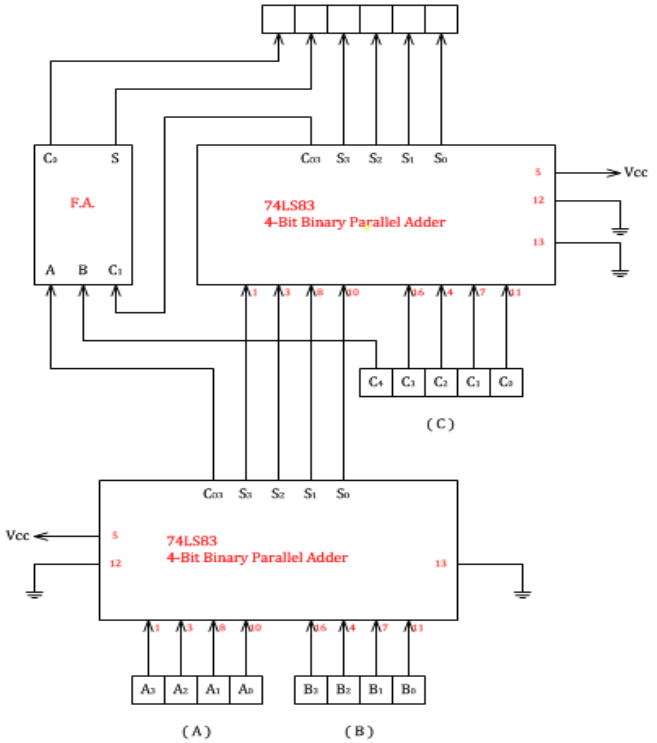
Rangkaian *adder* adalah suatu rangkaian *digital* yang melakukan penjumlahan penjumlahan bilangan dalam bentuk biner. *Adder* dapat dibentuk dengan rangkaian kombinasional saja atau gabungan rangkaian kombinasional dan sekuensial. Rangkaian kombinasional merupakan rangkaian yang kondisi keluarannya dipengaruhi oleh kondisi masukan, sedangkan rangkaian sekuensial merupakan rangkaian yang kondisi keluarannya dipengaruhi oleh kondisi masukan dan keadaan keluaran sebelumnya (bekerja berdasarkan urutan waktu). Salah satu rangkaian *adder* yang memanfaatkan penggabungan dengan rangkaian kombinasional ialah *Parallel Adder*.

Parallel adder adalah rangkaian penjumlah dari dua bilangan yang telah dikonversikan ke dalam bentuk biner. Rangkaian *Parallel adder* terdiri dari sebuah *half adder* (HA) pada *Least Significant Bit* (LSB) dari masing-masing input dan beberapa *full Adder* pada bit-bit berikutnya, atau bisa juga disusun atas tiga *full adder*. Berikut ini contoh rangkaian penjumlahan dari dua bilangan biner, rangkaian ini hanya membutuhkan 1 IC.



Gambar 3. 4 Rangkaian Dari Dua Penjumlahan Bilangan Biner

Berikut merupakan contoh rangkaian dari tiga penjumlahan bilangan biner.



Gambar 3. 5 Rangkaian Dari Tiga Penjumlahan Bilangan Biner

Untuk rangkaian *parallel adder* seperti pada *gambar 2*, diperlukan *combinational circuit* tambahan untuk mempersingkat waktu kerja dari *adder* tersebut, yang akan menghasilkan nilai *carry* (Cn, dengan nilai *n* tergantung pada urutan rangkaiannya). Diketahui pesamaan *full adder* adalah sebagai berikut.

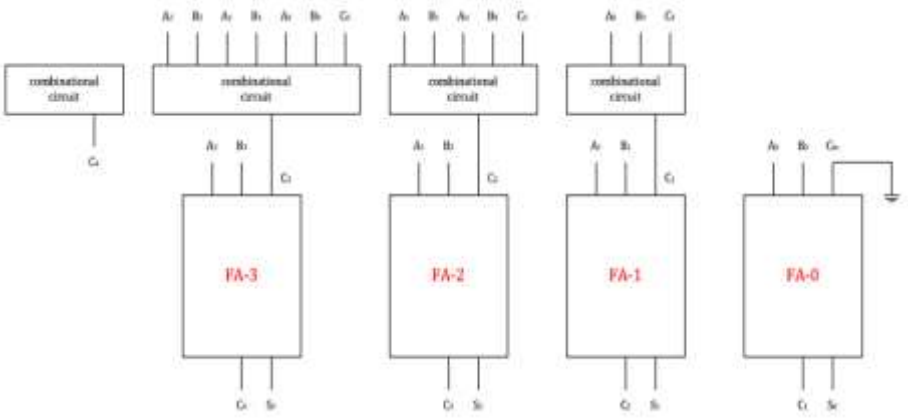
$$S = A \oplus B \oplus C_{in}$$

$$C0 = \overline{A}BC_{in} + A\overline{B}C_{in} + ABC_{in} + AB\overline{C_{in}}$$

$$= C_{in}(A \oplus B) + AB(C_{in} \oplus \overline{C_{in}})$$

$$= C_{in}(A \oplus B) + AB$$

maka, rangkaian *parallel adder* dengan waktu kerja yang dipersingkat akan menjadi seperti gambar di bawah ini.



Gambar 3. 6 Rangkaian Parallel Adder Dengan Waktu Kerja yang Dipersingkat

Rangkaian tersebut didapat dari persamaan berikut.

Persamaan 1 : $C_1 = A_0 B_0 + C_0 (A_0 B_0)$

Persamaan 2 : $C_2 = A_1 B_1 + C_1 (A_1 B_1)$

Persamaan 3 : $C_3 = A_2 B_2 + C_2 (A_2 B_2)$

Persamaan 4 : $C_4 = A_3 B_3 + C_3 (A_3 B_3)$

Jika *carry generator* (AB) disimbolkan dengan G dan *carry propagator* (AB) dilambangkan dengan P, persamaannya dapat dituliskan menjadi seperti ini.

Persamaan 1 : $C_1 = G_0 + C_0 \cdot P_0$

Persamaan 2 : $C_2 = G_1 + C_1 \cdot P_1$

Persamaan 3 : $C_3 = G_2 + C_2 \cdot P_2$

Persamaan 4 : $C_4 = G_3 + C_3 \cdot P_3$

Kemudian persamaan tersebut dijabarkan hingga menjadi persamaan-persamaan di bawah ini.

$C2 = G_1 + C_1 \cdot P_1$

$= G_1 + (G_0 + C_0 \cdot P0) P1$

$= G_1 + G_0 \cdot P_1 + C_0 \cdot P_0 \cdot P_1$

$C3 = G_2 + C_2 \cdot P_2$

$= G_2 + (G_1 + G_0 \cdot P_1 + C_0 \cdot P_0 \cdot P_1) P_2$

$= G_2 + G_1 \cdot P_2 + G_0 \cdot P_1 \cdot P_2 + C_0 \cdot P_0 \cdot P_1 \cdot P_2$

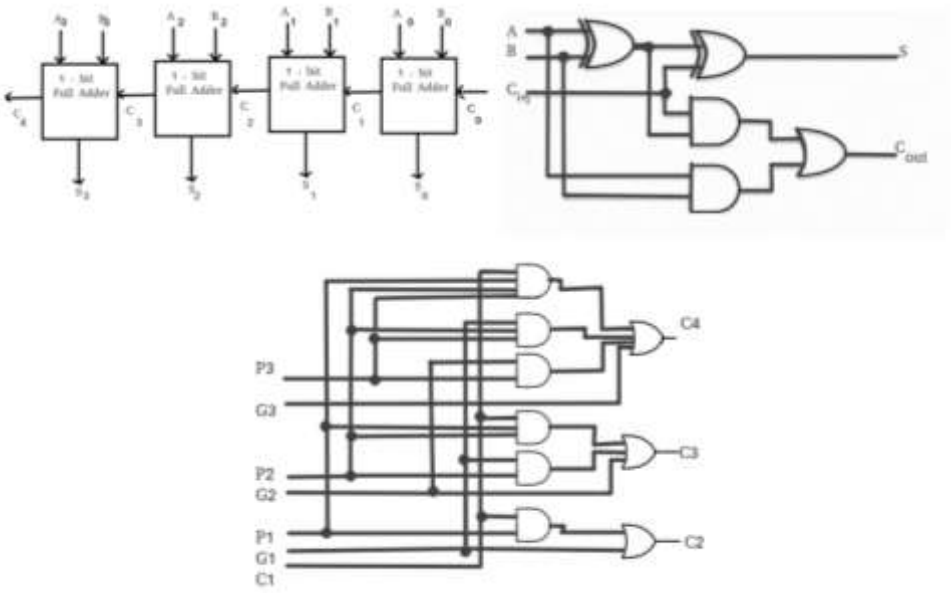
$$C4 = G_3 + C_3 \bullet P_3$$
$$= G_3 + (G_2 + G_1 \bullet P_2 + G_0 \cdot P_1 \bullet P_2 + C_0 \cdot P_0 \cdot P_1 \bullet P_2) P_3$$
$$= G_3 + G_2 \bullet P_3 + G_1 \cdot P_2 \bullet P_3 + G_0 \bullet P_1 \bullet G_0 \cdot P_2 \bullet P_3 + C_0 \cdot P_0 \cdot P_1 \bullet P_2 \bullet P_3$$

3.1.5 Rangkaian Carry Ahead Look Adder

Rangkaian Carry Look Ahead Adder (CLA) memiliki kesamaan dengan rangkaian Parallel Binary Adder (BPA), hal utama yang membedakan antara keduanya yaitu ketika pada rangkaian BPA perlu menunggu nilai C_0 sedangkan pada CLA hal tersebut tidak diperlukan.

Adder menghasilkan penundaan propagasi sambil melakukan operasi aritmatika lainnya seperti perkalian dan pembagian karena menggunakan beberapa langkah penambahan atau pengurangan. Ini adalah masalah utama bagi penambah dan karenanya meningkatkan kecepatan penambahan akan meningkatkan kecepatan semua operasi aritmatika lainnya. Oleh karena itu, mengurangi penundaan propagasi carry adder sangat penting. Ada berbagai pendekatan desain logika yang telah digunakan untuk mengatasi masalah propagasi pembawa. Salah satu pendekatan yang banyak digunakan adalah menggunakan carry look-ahead yang memecahkan masalah ini dengan menghitung sinyal carry terlebih dahulu, berdasarkan sinyal input. Jenis rangkaian adder ini disebut carry look-ahead adder.

Adder look-ahead carry mengurangi penundaan propagasi dengan memperkenalkan perangkat keras yang lebih kompleks. Dalam desain ini, desain pembawa riak diubah sedemikian rupa sehingga logika pembawa pada kelompok bit penambah tetap direduksi menjadi logika dua tingkat. Mari kita bahas desainnya secara detail.



Gambar 3. 7 Simbol dan Rangkaian Carry Ahead Adder

3.1.6 BCD Adder

Interface atau antarmuka tersebut biasanya akan menerima data dalam bentuk bilangan desimal kemudian rangkaian digital harus menggunakan beberapa kode biner agar dapat lebih mudah untuk mewakili bilangan desimal tersebut. Kode yang digunakan untuk tujuan ini biasanya disebut dengan kode BCD (*Binary Coded Decimal*). Dalam kode BCD, setiap bilangan Desimal diwakili oleh bilangan biner 4 bit. Jadi dapat dikatakan bahwa Kode BCD (*Binary Coded Decimal*) adalah *sistem pengkodean Biner dari angka Desimal dimana setiap digit Desimal diwakili oleh sejumlah bit, biasanya terdiri dari 4 bit*. Berikut merupakan rangkaian dari BCD adder.

Konversi BCD

Dalam proses konversinya, setiap angka dari bilangan desimal dikonversikan satu per satu menjadi 1 kelompok bilangan biner (4 bit) seperti contoh dibawah ini:

Pertama, pisahkan angka desimal sesuai dengan bobotnya kemudian tuliskan kode BCD yang ekuivalen untuk mewakili setiap digit desimal yang bersangkutan. Konversi Bilangan Desimal 489_{10} ke kode BCD:

4 = 0100

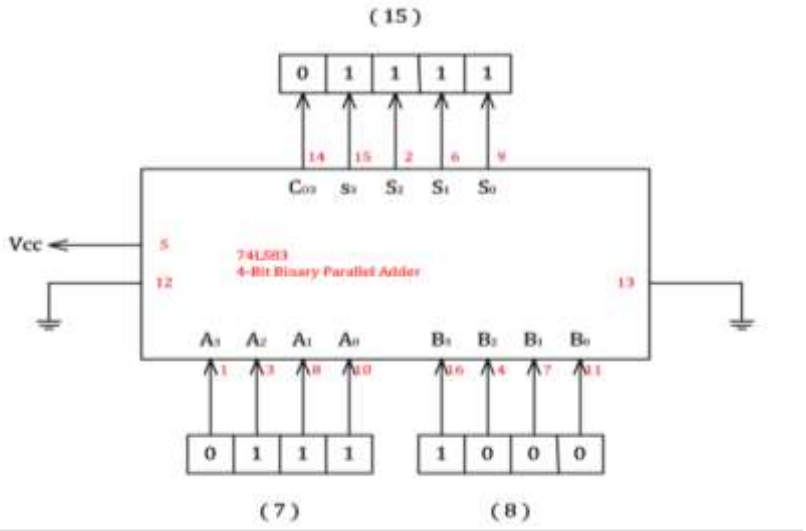
8 = 1000

9 = 1001

Bilangan desimal 48910 yang dikonversikan ke kode BCD adalah menjadi 0100 1000 1001 atau 010010001001 BCD.

3.1.7 Rangkaian Penjumlahan 2 Data 4 Bit.

Proses penjumlahan dimulai dengan menjumlahkan LSB dari augend (bilangan yang ditambah) dan addend (bilangan penambah). Jadi, 1 + 1 = 10, yang berarti bahwa sum untuk posisi ini sama dengan 0 dengan carry 1. Carry ini harus ditambahkan pada posisi berikutnya, yaitu dijumlahkan menjadi satu dengan augend dan addend pada posisi tersebut. Jadi, pada posisi kedua, 1 + 1 + 1 = 10, sum nya sama dengan 0 dan carry 1. Carry ini ditambahkan pada posisi berikutnya bersama-sama dengan bit-bit augend dan addend pada posisi itu, dan demikian seterusnya untuk posisi-posisi yang lainnya.



Gambar 3. 8 IC Rangkaian Parallel Binary Adder

Berdasarkan gambar di atas, dapat dilihat bahwa rangkaian tersebut menggunakan satu IC 7483 untuk membuat satu rangkaian *Parallel Binary Adder* dengan data penjumlahan sebanyak dua data. Dari rangkaian tersebut,

dicontohkan penjumlahan 7 + 8 yang berada di sisi *input* dengan hasil 15 pada sisi *output*. Cara menghitung nilai dari rangkaian full adder:

$A = A_3\ A_2$

$A_1\ A_0$

$B = B_3\ B_2$

$\underline{B_1\ B_0\ +}$

$Co = C_3\ C_2$

$C_1\ C_0$

Sedangkan untuk penjumlahan tiga data biner, dibutuhkan dua IC 7483 dan satu rangkaian *full adder*.

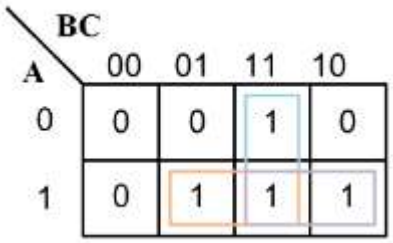
3.1.8 Contoh Aplikasi Rangkaian Decoder

1. Diketahui terdapat 3 saklar lampu, lampu akan menyala bila dua atau lebih saklar on. Tentukan tabel kebenaran, K-map dan Persamaannya!

- Tabel kebenaran:

Input			Output
A	B	C	L
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

- K-MAPS:



- Persamaan:

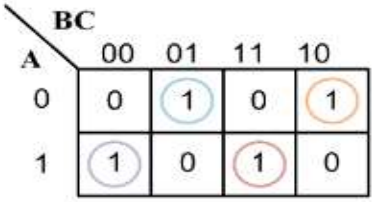
L = BC + AC + AB

2. Diketahui terdapat 3 saklar dan 1 lampu, lampu akan menyala jika seluruh saklar on atau statu saklar on, lampu akan padam jika seluruh saklar off atau dua saklar on. Tentukan tabel kebenaran, K-map dan Persamaannya!

- Tabel Kebenaran:

Input			Output
A	B	C	L
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

- K-MAPS



- Persamaan:

$$L = \overline{A\bar{B}}C + \bar{A}B\bar{C} + A\overline{B\bar{C}} + ABC$$

$$= \bar{A}(\bar{B}C + B\bar{C}) + A(\overline{B\bar{C}} + BC)$$

$$= \bar{A}(B \oplus C) + A(\overline{B \oplus C})$$

$$= A \oplus B \oplus C$$

3.1.9 Contoh Soal Mengenai Rangkaian Full Adder

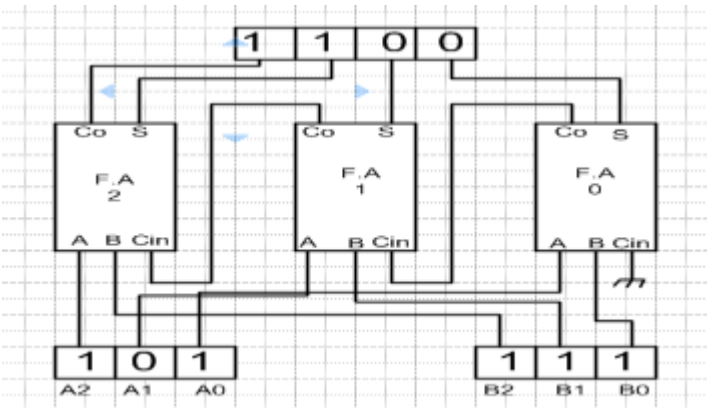
1. A = 101 (5), B = 111 (7) maka hasil penjumlahannya akan 12 setelah itu kita mencari pembuktian dengan menjumlahkan bilangan biner tersebut. Seperti berikut:

   101 + 111 = 1100 (12) berarti hasil penjumlahan biner tersebut terbukti benar. Kemudian disetiap biner pasti memiliki nilainya masing-masing seperti nilai biner paling kanan yang berarti A_0 dan juga pada nilai B yaitu B_0. Contoh perhitungan:

   $A_2\ A_1\ A_0$

   $\underline{B_2\ B_1\ B_0\ +}$

   Maka untuk rangkainnya seperti berikut :



Gambar 3. 9 Rangkaian Penjumlahan Full Adder

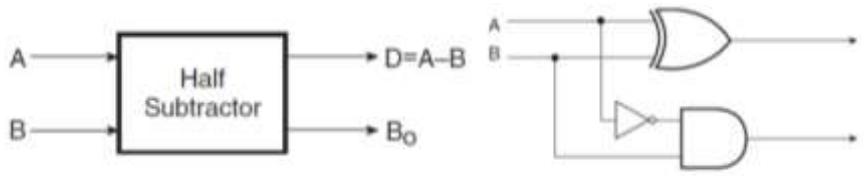
Apabila terdapat 3 bit dalam parallel binary adder maka hasilnya akan menjadi 4 kemudian apabila terdapat 4 bit maka hasilnya akan menjadi 5 bit.

3.2 Substractor

Substractor atau pengurang meimiliki cara kerja yang mirip dengan *adder*, hanya saja memiliki perbedaan hasli karena fungsi yang berbeda. *Adder* berfungsi sebagai penambah, sedangkan *subtractor* berfungsi sebagai pengurang.

Sama halnya dengan *adder*, *subtractor* pun dibagi menjadi dua, yaitu *half subtractor* (H.A.) dan *full subtractor* (F.A.).

3.2.1 Half Subtractor



Gambar 3. 10 Rangkaian Half Subtractor

Half subtractor adalah rangkaian pengurangan sistem bilangan biner yang paling sederhana, karena hanya dapat digunakan untuk operasi pengurangan data bilangan biner sampai 1 bit saja. Rangkaian ini memiliki 2 terminal input (a dan b) dan 2 terminal output (difference dan borrow). Rangkaian pengurang setengah atau half subtractor merupakan implementasi dari operasi pengurangan dasar dua bilangan biner. Pada rangkaian ini tidak ada pengurangan borrow in yang dilibatkan yang artinya pada rangkaian ini proses pengurangan belum sempurna.

Untuk membuat rangkaian *half subtractor*, dapat dilakukan dengan menggabungkan masing-masing satu gerbang EX-OR, gerbang NOT, dan gerbang AND. *Half subtractor* memiliki dua input/masukan (misal A dan B) dengan Bo (*borrow output*) dan D (*difference*) sebagai output/keluarannya. Hubungan D dengan inputnya adalah $D = A - B$.

Tabel kebenaran :

Input		Output	
A	B	Bo	D
0	0	0	0
0	1	1	1
1	0	0	1
1	1	0	0

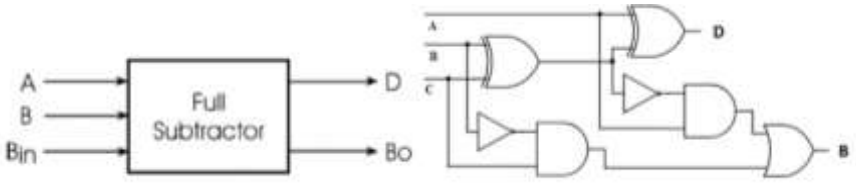
Persamaan :

$D = A\overline{B} + \overline{A}B$

$D = A \oplus B$

$Bo = \overline{A}B$

3.2.2 Full Substractor



Gambar 3. 11 Rangkaian Full Subtractor

Full subtractor adalah rangkaian yang digunakan untukn pengurangan bilangan-bilangan biner yang lebih dari 1 bit. Rangkaian ini terdiri dari 3 terminal input (a, b, dan carry-in) dan terminal output (difference dan borrow). Rangkaian full subtractor dibentuk dari 2 buah rangkaian half subtractor.

Sebuah full subtractor mengurangkan du bilangan yang telah dikonversikan menjadi bilangan-bilangan biner. Masing-masing bit pada posisi yang sama saling dikurangkan. Full subtractor mengurangkan dua bit input dan nilai borrow-out dari pengurangan bit sebelumnya output dari full subtractor adalah hasil pengurangan (remain) dan bit pinjamannya (borrow-out).

Sedangkan untuk membuat rangkaian *full subtractor*, dapat dilakukan dengan menggabungkan satu gebang NOT, dua gerbang NOR, dua gerbang EX-OR, dan tiga gerbang AND. *Full subtractor* memiliki tiga input (A, B, dan Bin/B input) dengan Bo (*borrow output*) dan D (*difference*) sebagai output/keluarannya. Hubungan D dengan inputnya adalah $D = A - B - B_{in}$.

Tabel kebenaran :

Input			Output	
A	B	Bin	Bo	D
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	1	0
1	0	0	0	1
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Persamaan :

$D = A \oplus B \oplus Bin$

$Bo = BBin + \overline{A}Bin + \overline{A}B$

3.3 Encoder

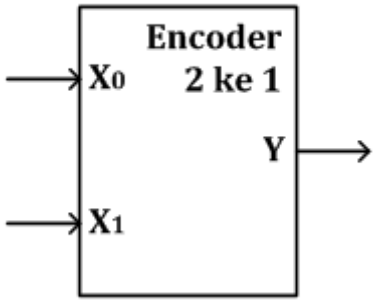
Encoder adalah rangkaian kombinasional yang mana merupakan operasi kebalikan dari decoder. Jika decoder mengubah input yang bentuknya kurang dimengerti manusia menjadi bentuk yang lebih familiar untuk manusia, maka encoder yang merupakan kebalikannya akan mengubah input yang dimengerti oleh manusia menjadi bentuk binary yang notabene bukan bahasa manusia. Sirkuit digital bernama encoder ini bisa dibilang merupakan integrated circuit (IC) dengan skala medium. Karena

pada dasarnya, Encoder meminimalkan jumlah saluran data dengan meringkasnya menjadi kode biner.

Prinsip cara kerja encoder yakni, dari semua input yang terpasang hanya satu kaki input yang diberikan logic 1 atau high dalam satu waktu. Dan encoder akan mengubah signal tersebut ke format kode bit-bit biner dengan tiap-tiap kaki input memiliki output kode tertentu. Encoder akan mengkodekan setiap jalur input yang aktif menjadi kode bilangan biner dengan konfigurasi 2^n input dan n output. Dia akan mengubah informasi dari 2^n input tersebut menjadi kode binary sejumlah n yang ekuivalen dengan inputnya.

3.3.1 Encoder 2 ke 1

Rangkaian a*Encoder* yang paling sederhana adalah *encoder* 2 ke 1, yaitu *encoder* yang hanya memiliki dua input/masukan dengan satu output/keluaran.

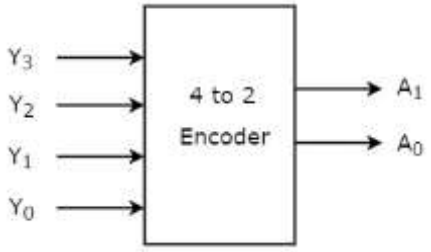
<table>
<tr><th>Simbol</th><th>Tabel Kebenaran</th><th>K-Map</th></tr>
<tr>
<td> 
</td>
<td>
<table>
<tr><td colspan="2">inp</td><td>out</td></tr>
<tr><td>X0</td><td>X1</td><td>Y</td></tr>
<tr><td>0</td><td>1</td><td>0</td></tr>
<tr><td>1</td><td>0</td><td>1</td></tr>
</table>
</td>
<td>
<table>
<tr><td>X0
X1</td><td>0</td><td>1</td></tr>
<tr><td>0</td><td>d</td><td>0</td></tr>
<tr><td>1</td><td>1</td><td>d</td></tr>
</table>
</td>
</tr>
</table>

Pada *encoder* 2 ke 1, yang menjadi input/masukan adalah X0 dan X1, dengan Y sebagai output/keluarannya. Dari tabel kebenaran, didapat persamaan $Y = X_1\overline{X_0}$. Kemudian, dalam bentuk K-Map (Karnough Map) ditemukan “d” yang disebut dengan *don’t care*, bisa ditulis jugan dengan 0 atau X. Simbol tersebut memiliki arti “yang tidak digunakan di dalam sistem”, muncul jika ada kombinasi input yang tidak digunakan dalam

sistem. Simbol *don't care* juga bisa dituliskan 0 atau 1 dengan memilih yang dirasa paling mengutungkan.

3.3.2 Encoder 4 ke 2

Encoder 4 to 2 ini terdiri dari 4 input Y3, Y2, Y1 dan Y0 dan dua output A1 dan A0. Di mana setiap salah satu dari input mendapat logic 1 maka akan menghasilkan kondisi kedua output tertentu. Berikut truth table atau tabel kebenaran dari tipe encoder 4 to 2 :



Gambar 3. 12 Simbol Encoder 4 ke 2

Tabel Kebenaran :

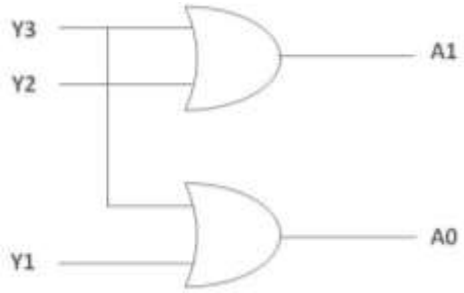
Input				Output	
Y3	Y2	Y1	Y0	A1	A0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

Jadi, persamaan logika untuk masing-masing output A1 dan A0 adalah :

A1 = Y3 + Y2

A0 = Y3 + Y1

Jika diimplementasikan ke dalam rangkaian logika menggunakan 2 gerbang OR, maka akan jadi rangkaian seperti di bawah ini:



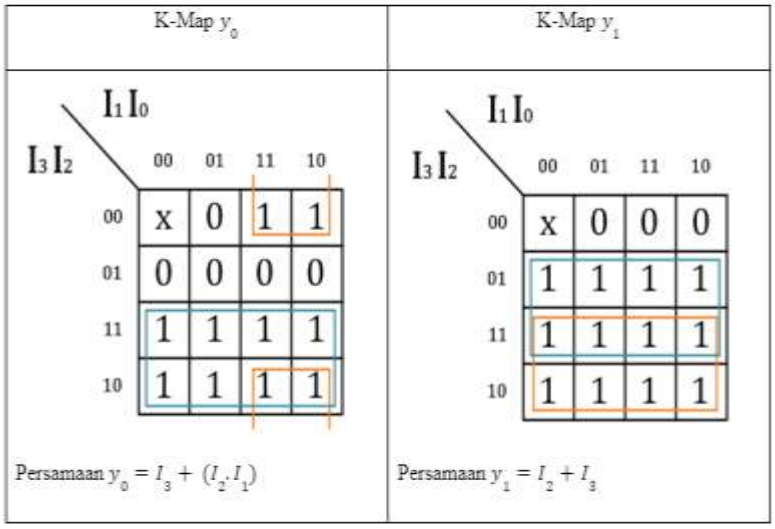
Gambar 3. 13 Rangkaian Encoder 4 ke 2

3.3.3 Priority Encoder 4 ke 2

Tabel kebenaran :

Input				Output	
I_3	I_2	I_1	I_0	Y_1	Y_0
0	0	0	1	0	0
0	0	1	X	0	1
0	1	X	X	1	0
1	X	X	X	1	1

Tanda "X" (*don't care*) muncul dikarenakan berapapun nilai dari input tersebut, nilai outputnya akan tetap sama seperti yang sudah tercantum dalam tabel kebenaran. Seperti yang sudah disebutkan sebelumnya, bahwa nilai output akan bergantung pada nilai input dengan prioritas yang lebih tinggi. Selain itu, nilai output bernilai biner sesuai dengan nilai I, misalnya output I_1 bernilai 01 karena 01 adalah biner dari angka 1. Dari tabel kebenaran tersebut, kita dapat mengetahui persamaannya lewat penyederhanaan dengan K-M.



Gambar 3. 14 K-Map Parity Encoder 4 to 2

3.3.4 Konversi Oktal ke Biner dengan Encoder 8 to 3

- Tabel Kebenaran:

Input (Oktal)		Output (Biner)		
I		Y_2	Y_1	Y_0
I_0	0	0	0	0
I_1	1	0	0	1
I_2	2	0	1	0
I_3	3	0	1	1
I_4	4	1	0	0
I_5	5	1	0	1
I_6	6	1	1	0
I_7	7	1	1	1

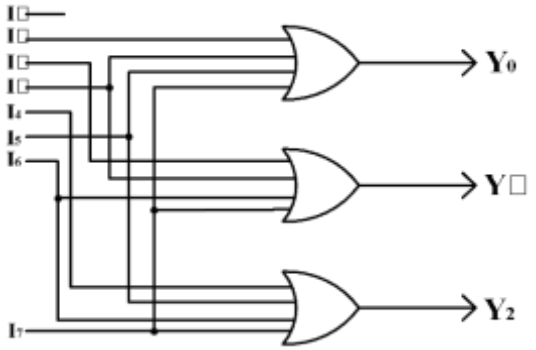
- Persamaan:

$Y_0 = I_1 + I_3 + I_5 + I_7$

$Y_1 = I_2 + I_3 + I_6 + I_7$

$Y_2 = I_4 + I_5 + I_6 + I_7$

- Rangkaian:



Gambar 3. 15 Rangkaian Konversi ke Biner dengan Encoder 8 to 3

3.3.5 Konversi Biner ke Oktal dengan Encoder 8 to 3

- Tabel Kebenaran:

Input			Output							
B_2	B_1	B_0	I_7	I_6	I_5	I_4	I_3	I_2	I_1	I_0
0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	0	0	0
1	0	0	0	0	0	1	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0

- Persamaan:

$I_0 = \overline{B0} \bullet \overline{B1} \bullet \overline{B2}$

$I_1 = B_0 \bullet \overline{B1} \bullet \overline{B2}$

$I_2 = \overline{B0} \bullet B_1 \bullet \overline{B2}$

$I_3 = B_0 \bullet B_1 \bullet \overline{B2}$

$I_4 = \overline{B0} \bullet \overline{B1} \bullet B_2$

$I_5 = B_0 \bullet \overline{B1} \bullet B_2$

$I_6 = \overline{B0} \bullet B_1 \bullet B_2$

$I_7 = B_0 \bullet B_1 \bullet B_2$

3.4 Multiplexer

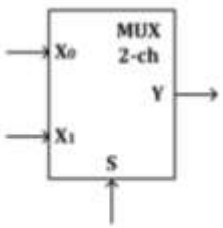
Multiplexer adalah rangkaian digital yang tersusun dari gerbang logika berkecepatan tinggi. Dimana perangkat tersebut merupakan komponen yang terdiri dari jalur input, terminal pengendali, dan juga jalur output.

Mux juga dikenal dengan istilah data selector atau perangkat pemilih data. Hal tersebut dikarenakan multiplexer bekerja seperti pada saklar rotary. Yaitu bertugas untuk memisahkan sinyal multiplex lalu kemudian memprosesnya dan menghubungkannya menjadi satu yaitu menuju jalur output. Menjadikan, sinyal yang masuk pada input, baik yang terdiri dari sinyal analog ataupun digital berkecepatan rendah nantinya akan digabungkan. Setelah itu baru bisa dipilih hingga kemudian ditransmisikan sehingga menjadi kombinasi logika dengan kecepatan yang lebih tinggi. Untuk tahapan selanjutnya yakni dapat dialihkan menuju jalur output sesuai dengan logika awal.

Singkatya, rangkaian *multiplexer* berfungsi memilih data yang ada pada masukan (X) untuk disalurkan pada keluarannya (Y), dengan bantuan dari sinyal pemilih atau sinyal control (S).

3.4.1 MUX – 2 Kanal

Multiplexer 2 kanal memiliki 2 nilai input yaitu X0 dan X serta memiliki 1 nilai output yaitu Y. Simbol, tabel kebenara dan penyederhanaan K-Map dari MUX 2 kanal sebagai berikut :



input			output
X1	X0	S	Y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

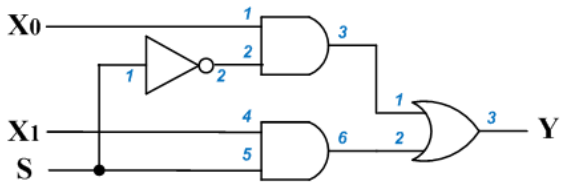
X1X0 \ S	0	1
00	0	0
01	1	0
11	1	1
10	0	1

Gambar 3. 16 Simbol, Tabel Kebenaran dan K-maps Mux 2 Kanal

Persamaan nilai MUX 2 kanal yaitu sebagai berikut :

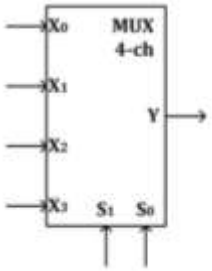
$$Y = SX_1 + \overline{S}X_0$$

Rangkaian MUX 2 kanal mengunakan IC 7408, 7404 dan 7432 sebagai berikut :



Gambar 3. 17 Rangkaian Mux 2 Kanal menggunakan IC 7408, 7404 dan 7432

3.4.2 MUX 4 kanal



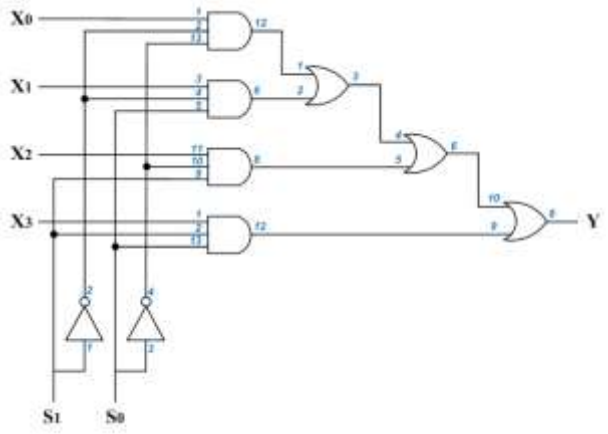
input		output
S1	S0	Y
0	0	X0
0	1	X1
1	0	X2
1	1	X3

Gambar 3. 18 Simbol dan Tabel Kebenaran 4 Kanal

Multiplexer 4 kanal memiliki 4 nilai input yaitu X0, X1, X2 dan X3 serta memiliki 1 nilai output yaitu Y. Persamaan nilai MUX 4 kanal yaitu sebagai berikut :

$$Y = \overline{S_1}\,\overline{S_0}X_0 + \overline{S_1}\,S_0X_1 + S_1\,\overline{S_0}X_2 + S_1S_0X_3$$

Berikut ini rangkaian dari MUX 4 kanal.



Gambar 3. 19 Rangkaian Mux 4 Kanal

3.4.3 Aplikasi Rangkaian Multiplexer

Aplikasi tambahan *multiplexer* dicontohkan dengan MUX 2 ke 1. Dari persamaan $G = A\overline{B}C + \overline{A}\,\overline{B}C + ABC + \overline{A}\,\overline{B}\,\overline{C}$, maka didapat penyelesaian sebagai berikut apabila A berlaku sebagai *selector* (nilai A dikeluarkan)

$$G = A\left(\overline{B}C + BC\right) + \overline{A}\left(\overline{B}C + \overline{B}\,\overline{C}\right)$$

$$G = A\left(C\left(\overline{B} + B\right)\right) + \overline{A}\left(\overline{B}\left(C + \overline{C}\right)\right)$$

$$G = A(C) + \overline{A}(\overline{B})$$

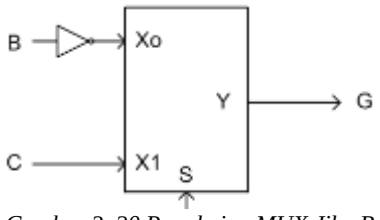
Implementasikan persamaan logika berikut menggunakan gerbang-gerbang dasar logika :

$$G = A\overline{B}C + \overline{ABC} + ABC + \overline{ABC}$$

Multiplexer 2 to 1 (2 kanal)

Tabel kebenaran dan Rangkaian Multiplexer jika input B menjadi selectornya :

Input			Output
B	A	C	G
0	0	0	0
0	0	1	1
0	1	0	0



Gambar 3. 20 Rangkaian MUX Jika B Menjadi Selector

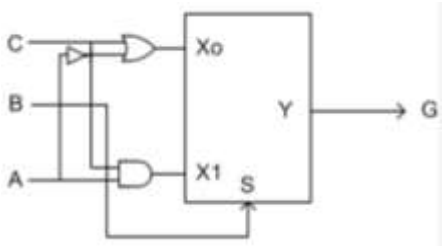
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

S Xo X1

Tabel kebenaran dan Rangkaian Multiplexer jika input A menjadi selectornya :

Input			Output
A	B	C	G
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1

S Xo X1



Gambar 3. 21 Rangkaian MUX Jika A Menjadi Selector

Jika menggunakan cara lain, mengunakan K – map

Implementasikan persamaan logika berikut menggunakan gerbang logika dasar

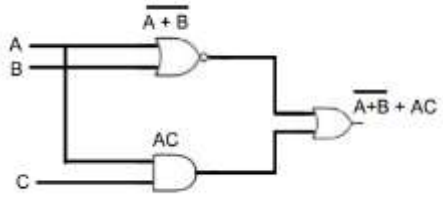
G = $A\bar{B}C + \bar{A}\bar{B}C + ABC + \bar{A}\bar{B}\bar{C}$

G = $A(\bar{B}C + B\,C) + A(B\,C + B\,\bar{C})$

G = $A\,(C\,(\bar{B} + B\,) + A\,(\bar{B}\,(C + \bar{C})$

G = A (C) + $\bar{A}(\bar{B})$

Gambar rangkaian :



Gambar 3. 22 Rangkaian Mux 2 Kanal Jika Menggunakan K-Map

Jika memakai K map :

A/BC	00	01	11	10
0	1	1	0	0
1	0	1	1	0

Persamaan yang di dapat :

G = $\overline{A}\,\overline{B} + A\,C$

3.5 DeMultiplexer

Demultiplexer adalah rangkaian logika yang menerima satu input data dan mendistribusikan input tersebut ke beberapa output yang tersedia. Kendali pada demultiplekser akan memilih saklar mana yang akan dihubungkan. Pemilihan keluarannya dilakukan melalui masukan penyeleksi.

Seleksi data-data input dilakukan oleh selector line, yang juga merupakan input dari demultiplexer tersebut. Pada demultiplexer saluran kendali sebanyak "n" saluran dapat menyeleksi saluran keluaran. Pada demultiplexer, masukan data dapat terdiri dari beberapa bit. Keluarannya terdiri dari beberapa jalur, masing-masing jalur terdiri dari satu atau lebih dari satu bit. Masukan selector terdiri dari satu atau lebih dari satu bit tergantung pada banyaknya jalur keluaran.

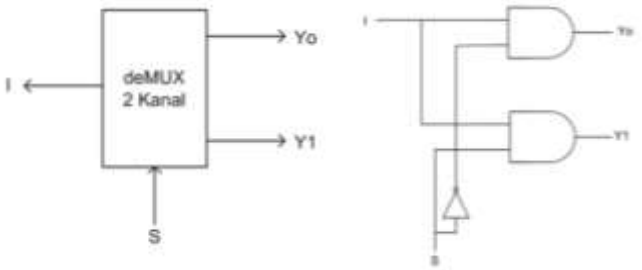
Terdapat beberapa jenis, namun berikut merupakan jenis deMUX yang sering digunakan :

3.5.1 deMUX – 2 Kanal

deMultiplexer 2 kanal memiliki 2 nilai input yaitu I dan S, Serta memiliki 2 nilai output yaitu Yo dan Y1. Persamaan nilai deMUX 2 kanal yaitu sebagai berikut :

$Yo = \overline{S}I$

$Y1 = SI$



Gambar 3. 23 Rangkaian DeMux 2 Kanal

Rangkaian yang disusun pada protoboard sama dengan rangkaian pada gambar berikut. Pada rangkaian tersebut memerlukan dua jenis IC yaitu IC AND dengan nomor 7408, dan IC NOT dengan nomor 7404.

3.5.2 deMUX 4 kanal

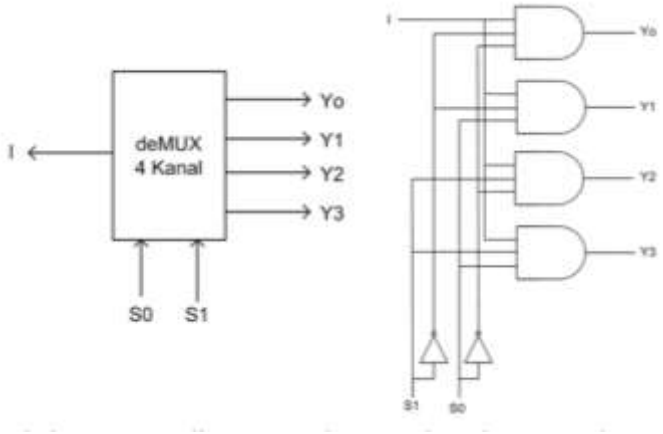
deMultiplexer 4 kanal memiliki 3 nilai input yaitu I, S0 dan S1, Serta memiliki 4 nilai output yaitu Yo, Y1, Y2 dan Y3. Persamaan nilai deMUX 4 kanal yaitu sebagai berikut :

$Yo = \overline{S1S0}I$

$Y1 = \overline{S1}S0I$

$Yo = S1\overline{S0}I$

$Yo = S1S0I$

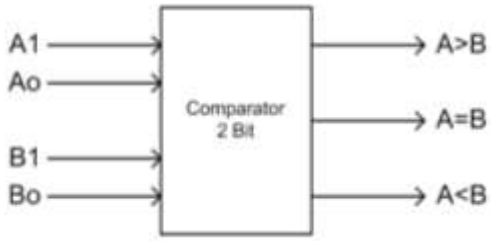


Gambar 3. 24 Rangkaian DeMux 4 Kanal

Rangkaian yang disusun pada protoboard sama dengan rangkaian deMultiplexer 4 kanal pada gambar berikut. Sirkuit ini terbentuk dari susunan gerbang logika, IC triple AND dengan nomor 7411, dan IC NOT dengan nomor 7404. Pada input terdiri atas gerbang S1, S0 dan I. Sedangkan pada gerbang output terdiri atas gerbang Yo, Y1, Y2 dan Y3.

3.6 Comparator

Rangkaian Comparator adalah satu jenis penerapan rangkaian kombinasional yang mempunyai fungsi utama membandingkan dua data digital. Hasil pembandingan itu adalah, sama, lebih kecil, atau lebih besar. Dari dua data digital yang hanya terdiri dari 1 bit yang dibandingkan, kemudian dapat diperluas menjadi dua data digital yang terdiri dari lebih dari 1 bit seperti dua bit, tiga bit, dst. Komparator banyak digunakan misalnya pada mesin penyeleksi surat, baik ukuran dimensinya, berat surat, kode area (berdasarkan bar-code), dsb. Berikut contoh Gambaran rangkaian komparator 1-bit.



Gambar 3. 25 Rangkaian Comparator 2 Bit

Misalkan kita ingin merancang suatu alat pembanding (comparator) yang akan membandingkan dua angka dan memberkan hasilnya, yaitu angka yang satu lebih kecil, lebih besar, atau sama dengan angka yang satunya. Sistem pembanding ini digambarkan secara garis besar sebagai sebuah kotak hitam yang hanya diketahui fungsinya saja.

Sistem pembanding ini mempunyai 2 Input A dan B yang masing-masing terdiri dan 2 bit dan 3 output yang masing-masing terdiri dari 1 bit untuk menunjukkan hasil perbandingan tersebut yaitu, A>B, A<B, dan A=B. Cara kerja sistem ini sangatlah sederhana. Setiap waktu hanya ada satu output yang bernilai BENAR. Output A>B akan bernilai ‘1’ apabila nilai A lebih besar dari B. Demikian juga halnya dengan output A<B dan A=B yang bernilai ‘1’ apabila nilai A lebih kecil dari B dan apabila nilai A sama dengan B.

Sistem ini akan mempunyai 3 persamaan logika karena adanya 3 output. Oleh karena itu kita akan sederhanakan dan peroleh persamaan logikanya satu persatu. Jika diperhatikan, persamaan logika dari ketiga output tersebut dinyatakan dalam 4 variabel inputnya yaitu A1, A0, B1, dan B0. Hal ini menunjukkan bahwa setiap outputnya tergantung pada input-inputnya. Di dalam mendesain sistem pembanding yang sebenarnya dengan menggunakan komponen-komponen digital, kita ingin berusaha untuk mengurangi jumlah ICs/komponen yang digunakan. Suatu penghematan yang jelas dan mudah di peroleh dengan mengamati persamaan-persamaan logika yang di peroleh adalah dengan adanya kanonical term yang sama di antara persamaan persamaan logika tersebut. Sebagai contohnya dalam

desain sistem pembanding ini ialah kanonikal term A0.A1.B0 yang terdapat pada persamaan logika untuk output A > B dan A < B. Hal ini berarti bahwa hanya satu rangkaian yang perlu dibangun untuk kanonikal term ini sehingga output A > B dan A < B akan menggunakannya bersama.

Perlu diingat juga bahwa pada sistem ini hanya akan ada satu output yang akan bernilai BENAR=1 untuk setiap kombinasi inputnya; sebagai contohnya untuk input 01 (A1 & A0) dan 11 (B1 & B0) hanya output A < B yang akan bernilai BENAR=1. Dengan menyadari hal semacam ini, maka akan menolong kita untuk mengetahui apabila sistem tersebut tidak bekerja dengan semestinya misalnya jika output A < B dan A = B memberikan nilai BENAR untuk contoh input di atas tadi.

BAB 4
SEVEN SEGMENT DISPLAY

4.1 Pengertian Seven Segment Display

Seven Segment Display (7 Segment Display) dalam bahasa Indonesia disebut dengan Layar Tujuh Segmen adalah komponen Elektronika yang dapat menampilkan angka desimal melalui kombinasi-kombinasi segmennya. *Seven Segment Display* pada umumnya dipakai pada Jam Digital, Kalkulator, Penghitung atau Counter Digital, Multimeter Digital dan juga Panel Display Digital seperti pada Microwave Oven ataupun Pengatur Suhu Digital . *Seven Segment Display* pertama diperkenalkan dan dipatenkan pada tahun 1908 oleh Frank. W. Wood dan mulai dikenal luas pada tahun 1970-an setelah aplikasinya pada LED (Light Emitting Diode).

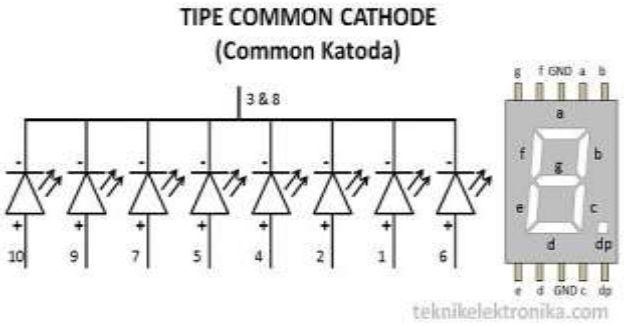
Seven Segment Display memiliki 7 Segmen dimana setiap segmen dikendalikan secara ON dan OFF untuk menampilkan angka yang diinginkan. Angka-angka dari 0 (nol) sampai 9 (Sembilan) dapat ditampilkan dengan menggunakan beberapa kombinasi Segmen. Selain 0 – 9, *Seven Segment Display* juga dapat menampilkan Huruf Hexadecimal dari A sampai F. Segmen atau elemen-elemen pada Seven Segment Display diatur menjadi bentuk angka "8" yang agak miring ke kanan dengan tujuan untuk mempermudah pembacaannya. Pada beberapa jenis Seven Segment Display, terdapat juga penambahan "titik" yang menunjukan angka koma decimal. Terdapat beberapa jenis *Seven Segment Display*, diantaranya adalah Incandescent bulbs, Fluorescent lamps (FL), Liquid Crystal Display (LCD) dan Light Emitting Diode (LED).

4.2 Jenis – Jenis Seven Segment Display

1. Seven Segment Common Cathode

   Pada LED 7 Segmen jenis Common Cathode (Katoda), Kaki Katoda pada semua segmen LED adalah terhubung menjadi 1 Pin, sedangkan Kaki Anoda akan menjadi Input untuk masing-masing Segmen LED. Kaki

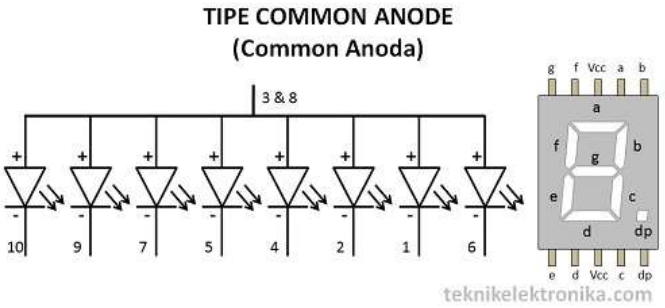
Katoda terhubung pada Terminal Negatif (-) atau Ground sedangkan Signal Kendali (Control Signal) akan diberikan kepada masing-masing Kaki Anoda Segmen LED.



Gambar 4. 1 Rangkaian Common Cathode

2. Seven-Segment Common Anoda

Pada LED 7 Segmen jenis Common Anode (Anoda), Kaki Anoda pada semua segmen LED adalah terhubung menjadi 1 Pin, sedangkan kaki Katoda akan menjadi Input untuk masing-masing Segmen LED. Kaki Anoda yang terhubung menjadi 1 Pin ini akan diberikan Tegangan Positif (+) dan Signal Kendali (control signal) akan diberikan kepada masing-masing Kaki Katoda Segmen LED.



Gambar 4. 2 Rangkaian Common Anode

4.3 Prinsip Kerja Dasar



Gambar 4. 3 Blok Dasar Seven Segment Diplay

Blok Dekoder pada diagram diatas mengubah sinyal Input yang diberikan menjadi 8 jalur yaitu “a” sampai “g” dan poin decimal (koma) untuk meng-ON-kan segmen sehingga menghasilkan angka atau digit yang diinginkan. Contohnya, jika output dekoder adalah a, b, dan c, maka Segmen LED akan menyala menjadi angka “7”.

Jika Sinyal Input adalah berbentuk Analog, maka diperlukan ADC (Analog to Digital Converter) untuk mengubah sinyal analog menjadi Digital sebelum masuk ke Input Dekoder. Jika Sinyal Input sudah merupakan Sinyal Digital, maka Dekoder akan menanganinya sendiri tanpa harus menggunakan ADC.

Fungsi daripada Blok Driver adalah untuk memberikan arus listrik yang cukup kepada Segmen/Elemen LED untuk menyala. Pada Tipe Dekoder tertentu, Dekoder sendiri dapat mengeluarkan Tegangan dan Arus listrik yang cukup untuk menyalakan Segmen LED maka Blok Driver ini tidak diperlukan. Pada umumnya Driver untuk menyalakan 7 Segmen ini adalah terdiri dari 8 Transistor Switch pada masing-masing elemen LED.

4.4 BCD to Seven Segment Display

IC Decoder Digital, adalah perangkat yang mengubah satu format digital menjadi format digital lainnya dan salah satu perangkat yang paling umum digunakan untuk melakukan ini disebut Binary Coded Decimal (BCD) to 7-Segment Display Decoder.

Display tipe 7-segment LED (Light Emitting Diode) atau LCD (Liquid Crystal Display), menyediakan cara yang sangat nyaman untuk menampilkan informasi atau data digital dalam bentuk angka, huruf atau bahkan karakter alfanumerik.

Biasanya display 7-segmen terdiri dari tujuh LED berwarna individu (disebut segmen), dalam satu paket display tunggal. Untuk menghasilkan angka atau karakter HEX yang diperlukan dari 0 hingga 9 dan A hingga F , pada tampilan kombinasi yang benar dari segmen LED perlu menyala dan Dekoder Tampilan BCD ke 7-segmen seperti 74LS47 melakukan hal itu.

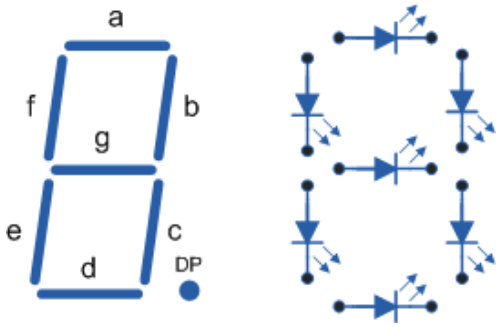
Layar LED 7-segmen standar umumnya memiliki delapan (8) koneksi input, satu untuk setiap segmen LED dan satu yang berfungsi sebagai terminal umum atau koneksi untuk semua segmen layar internal. Beberapa tampilan tunggal juga memiliki pin input tambahan untuk menampilkan titik desimal di pojok kanan bawah atau kiri.

Dalam elektronika ada dua jenis tampilan digital LED 7-segmen yang penting.

1. Tampilan Katoda Umum (CCD) – Pada tampilan katoda umum, semua koneksi katoda LED digabungkan menjadi logika “0” atau ground. Segmen individu diterangi oleh penerapan sinyal "TINGGI", logika "1" ke terminal Anoda individu.
2. Tampilan Anoda Umum (CAD) – Dalam tampilan anoda umum, semua sambungan anoda dari LED digabungkan menjadi logika “1” dan masing-masing segmen diterangi dengan menghubungkan terminal Katoda individu ke logika “RENDAH”. sinyal “0”.

Sambungan listrik dari masing-masing dioda untuk tampilan katoda umum dan tampilan anoda umum dan dengan menerangi setiap dioda pemancar cahaya secara terpisah, mereka dapat dibuat untuk menampilkan berbagai angka atau karakter.

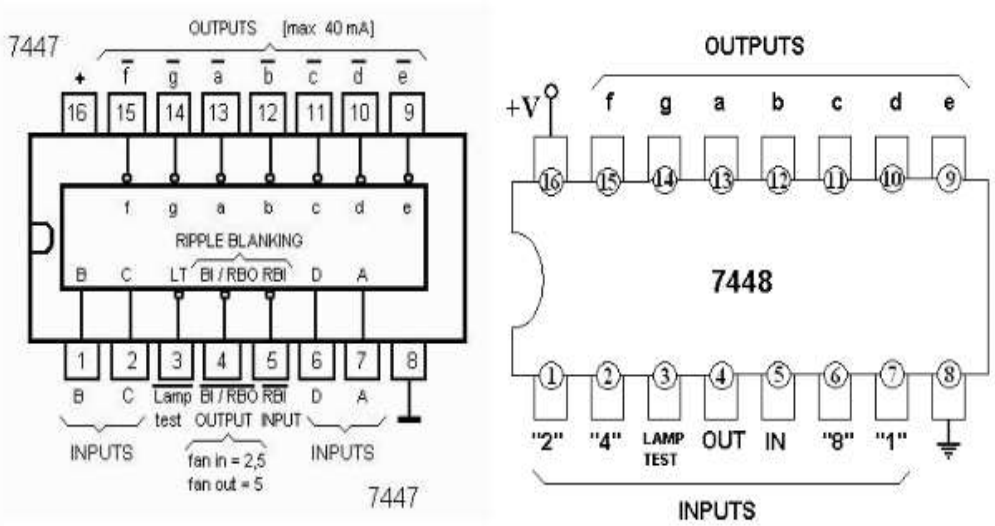
Format decoder tampilan 7 segment:



Gambar 4. 4 Format decoder tampilan 7 segment

Jadi untuk menampilkan angka "3" misalnya, segmen a , b , c , d dan g perlu disinari. Jika kita ingin menampilkan angka atau huruf yang berbeda, maka rangkaian segmen yang berbeda perlu disinari. Kemudian untuk tampilan 7-segmen, kita dapat menghasilkan tabel kebenaran yang memberikan segmen-segmen yang perlu disinari agar menghasilkan karakter yang dibutuhkan.

Dalam seven segment juga membutuhkan IC agar dapat menghasilkan angka lebih dari 8 yaitu IC 7447 dan IC 7448. Berikut merupakan data sheet IC 7447 dan IC 7448 :



Gambar 4. 5 IC BCD to Seven Segment Display

4.5 Contoh Soal BCD to Seven Segment dengan Decoder Sederhana

1) Rancanglah rangkaian decoder “biner to octal” menggunakan tampilan seven segment active low.

Tahapan Pengerjaan:

- Tabel Kebenaran:

Input Biner			Output Oktal Tampilan 7 Segment						
B_2	B_1	B_0	a	b	c	d	e	f	g
0	0	0	0	0	0	0	0	0	1
0	0	1	1	0	0	1	1	1	1
0	1	0	0	0	1	0	0	1	0
0	1	1	0	0	0	0	1	1	0
1	0	0	1	0	0	1	1	0	0
1	0	1	0	1	0	0	1	0	0
1	1	0	0	1	0	0	0	0	0
1	1	1	0	0	0	1	1	1	1

- Buatlah Persamaan Minterm (Persamaan Tabel Kebenaran) nya untuk output “a”, “b”, …, “g”.

a = ∑(m1,m4,m6)

B2 \ B1 B0	00	01	11	10
0	0	1	0	0
1	1	0	0	0

Persamaan: $\overline{B1}(B2 \oplus B0)$

b = ∑(m5,m6)

B2 \ B1 B0	00	01	11	10
0	0	0	0	0
1	0	1	0	1

Persamaan: $B2(B1 \oplus B0)$

c = ∑(m2)

B2 \ B1 B0	00	01	11	10
0	0	0	0	1
1	0	0	0	0

Persamaan: $\overline{B2}B1\overline{B0}$

d = ∑(m1,m4,m7)

B2 \ B1 B0	00	01	11	10
0	0	1	0	0
1	1	0	1	0

Persamaan; $\overline{B2B1}B0 + B2(\overline{B1B0} + B1B0)$

e = ∑(m1,m3, m4,m5,m7)

B2 \ B1 B0	00	01	11	10
0	0	1	1	0
1	1	1	1	0

Persamaan: $B0 + B2\overline{B1}$

f = ∑(m1,m2,m3,m7)

B2 \ B1 B0	00	01	11	10
0	0	1	1	1
1	0	0	1	0

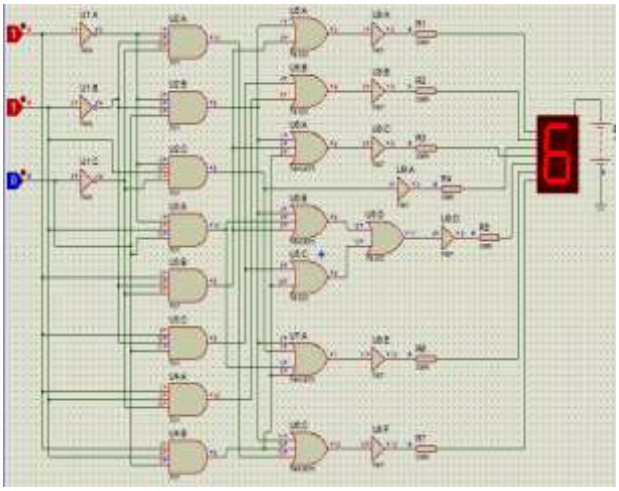
Persamaan: $\overline{B2}(B0 + B1) + B1B0$

g = ∑(m0,m1m7)

B2 \ B1 B0	00	01	11	10
0	1	1	0	0
1	0	0	1	0

Persamaan; $\overline{B2B1} + B2B1B0$

- Gambar Rangkaian:



Gambar 4. 6 BCD to Seven Segment dengan Decoder Sederhana Pada Proteous

2) Rancanglah rangkaian dekoder to 7 segment active high, yang mempunyai input BCD dan 6 output sebagai berikut :

BCD			
B_2	B_1	B_0	Output
0	0	0	0
0	0	1	1
0	1	0	2
0	1	1	3
1	0	0	4
1	0	1	5

Active High → Common Cathode

Active Low → Common Anode

Penyelesaian :

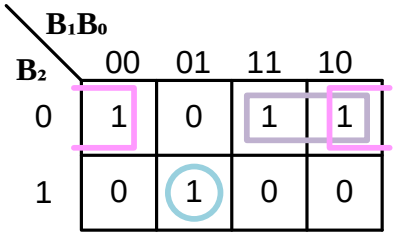
- Tabel Kebenaran

Input			Output 7 Segment						
B_2	B_1	B_0	a	b	c	d	e	f	g
0	0	0	1	1	1	1	1	1	0
0	0	1	0	1	1	0	0	0	0
0	1	0	1	1	0	1	1	0	1
0	1	1	1	1	1	1	0	0	1
1	0	0	0	1	1	0	0	1	1
1	0	1	1	0	1	0	1	1	1

- Karnaugh-Map dan Persamaan Output.

Membuat Persamaan paling sederhana dari tabel kebenaran tersebut menggunakan K-Map.

a = $\sum(m_0, m_2, m_3, m_5)$

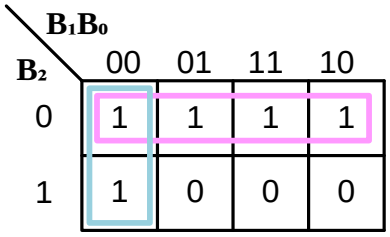


B_2 \ B_1B_0	00	01	11	10
0	1	0	1	1
1	0	1	0	0

a = $\overline{B_2}\,\overline{B_0} + \overline{B_2}B_1 + B_2\overline{B_1}B_0$

= $\overline{B_2}(\overline{B_0} + B_1) + B_2\overline{B_1}B_0$

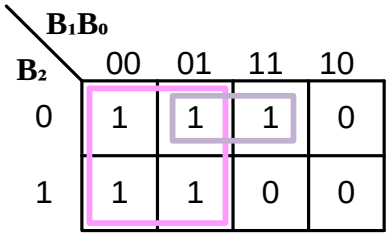
b = $\sum(m_0, m_1, m_2, m_3, m_4)$



B_2 \ B_1B_0	00	01	11	10
0	1	1	1	1
1	1	0	0	0

b = $\overline{B_2} + \overline{B_1}\,\overline{B_0}$

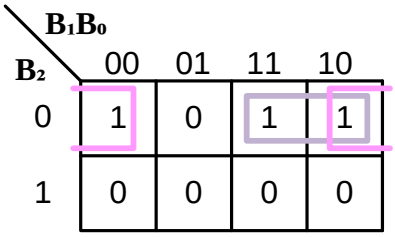
c = $\sum(m_0, m_1, m_3, m_4, m_5)$



B_2 \ B_1B_0	00	01	11	10
0	1	1	1	0
1	1	1	0	0

c = $\overline{B_1} + \overline{B_2}B_0$

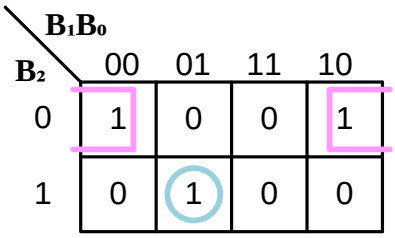
d = $\sum(m_0, m_2, m_3)$



B_2 \ B_1B_0	00	01	11	10
0	1	0	1	1
1	0	0	0	0

d = $\overline{B_2}\,\overline{B_0} + \overline{B_2}B_1$

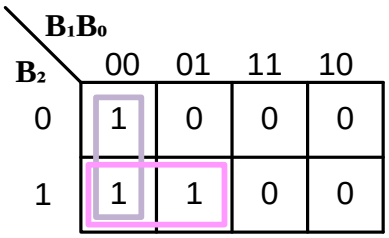
e = $\sum(m_0, m_2, m_5)$



B_2 \ B_1B_0	00	01	11	10
0	1	0	0	1
1	0	1	0	0

e = $\overline{B_2}\,\overline{B_0} + B_2\overline{B_1}B_0$

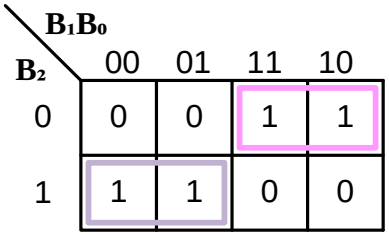
f = $\sum(m_0, m_4, m_5)$



B_2 \ B_1B_0	00	01	11	10
0	1	0	0	0
1	1	1	0	0

f = $\overline{B_1}\,\overline{B_0} + B_2\overline{B_1}$

g = $\sum(m_2, m_3, m_4, m_5)$



B_2 \ B_1B_0	00	01	11	10
0	0	0	1	1
1	1	1	0	0

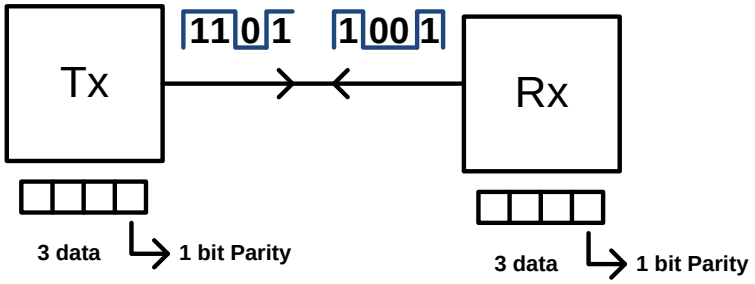
g= $\overline{B_2}B_1 + B_2\overline{B_1} = B_2 \oplus B_1$

BAB 5
PARITY GENERATOR DAN PARITY CHECKER

5.1 Pengertian Parity Generator dan Parity Checker

Bit parity adalah bit tambahan yang ditambahkan ke aliran data untuk memastikan keamanan transmisi. Paritas ini biasanya ditambahkan ke akhir string data dan dapat digunakan untuk mendeteksi kesalahan transmisi.

Bit paritas yang umum digunakan ada dua jenis, yaitu paritas genap dan paritas ganjil. Dengan paritas genap, paritas yang dihasilkan adalah 1 jika jumlah bit bernilai 1 dalam data yang dikirimkan ganjil, dan sebaliknya. Sedangkan untuk paritas ganjil, paritas yang dihasilkan adalah 1 jika jumlah bit bernilai 1 dalam data yang dikirimkan genap, dan sebaliknya. Contoh penggunaan parity bit adalah pada transmisi data melalui jaringan komputer atau saluran komunikasi lainnya. Saat data dikirim, paritas ditambahkan ke akhir data dan kemudian dikirim dengan data tersebut. Penerima kemudian dapat menggunakan paritas ini untuk mendeteksi jika ada kesalahan transmisi dalam data yang diterima. Jika bit paritas tidak sesuai dengan data yang diterima, diketahui telah terjadi kesalahan transmisi dan data harus ditransmisikan ulang.



Gambar 5. 1 Rangkaian Parity Generator dan Parity Checker

Bit tambahan yang dikirimkan oleh transmitter sebagai upaya agar reciver dapat mengetahui ada tidaknya terjadinya eror.

Generator parity adalah sirkuit yang digunakan untuk menghasilkan atau menghasilkan bit paritas. Paritas ini dihasilkan dari string data yang

terdiri dari sekumpulan bit biner. Paritas dihasilkan sebelum data dikirim atau dikirim ke penerima. Oleh karena itu generator paritas harus ditempatkan di sisi pengirim atau pengirim.

Pemeriksa paritas adalah sirkuit yang digunakan untuk memeriksa bit data yang dihasilkan oleh generator paritas dan urutan bit paritas setelah dikirim atau dikirim ke penerima. Pemeriksaan paritas mengembalikan nilai 0 atau 1, menunjukkan kesalahan pada bit yang diterima. Jika bendera kesalahan adalah 1, bit yang diterima salah. Sebaliknya, jika nilainya 0, berarti bit yang diterima benar. Pemeriksaan paritas ini ada di sisi penerima (RX).

5.2 Jenis – Jenis Parity Generator dan Parity Checker

1. Odd Parity

   → Jumlah angka 1 yang dikirimkan berjumlah ganjil.

   Contoh Soal :

1) Buatlah rangkaian Parity Generator untuk 3 bit informasi dengan jenis “Odd Parity”.

   Penyelesaian :

   - Tabel Kebenaran

Data			Parity
A	B	C	D
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

- Karnaugh-Map

Membuat Persamaan paling sederhana dari tabel kebenaran tersebut menggunakan K-Map.

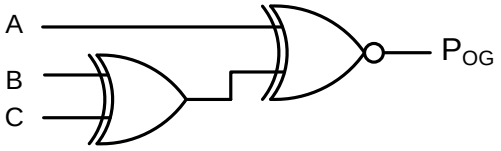
A \ BC	00	01	11	10
0	1	0	1	0
1	0	1	0	1

- Persamaan Output

Berdasarkan K-Map tersebut, persamaan yang dihasilkan adalah sebagai berikut :

$$P_{OG} = \overline{A}\left(\overline{BC} + BC\right) + A(\overline{B}C + BC)$$
$$= \overline{A}\overline{(B \oplus C)} + A(B \oplus C)$$
$$= \overline{A\overline{X} + AX}$$
$$= \overline{A \oplus X}$$
$$= \overline{A \oplus (B \oplus C)}$$

- Rangkaian



Gambar 5. 2 Rangkaian PoG

2) Buatlah Rangkaian Parity Checker untuk 3 bit informasi dengan jenis "Odd Parity".

Penyelesaian :

- Tabel Kebenaran

Data			Parity	Parity Error Checker
A	B	C	D	P_{EC}
0	0	0	0	1
0	0	0	1	0
0	0	1	0	0

0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	0
1	0	0	0	0
1	0	0	1	1
1	0	1	0	1
1	0	1	1	0
1	1	0	0	1
1	1	0	1	0
1	1	1	0	0
1	1	1	1	1

- Karnaugh-Map

Membuat Persamaan paling sederhana dari tabel kebenaran tersebut menggunakan K-Map.

AB \ CD	00	01	11	10
00	(1)	0	(1)	0
01	0	(1)	0	(1)
11	(1)	0	(1)	0
10	0	(1)	0	(1)

- Persamaan Output

Berdasarkan K-Map tersebut, persamaan yang dihasilkan adalah sebagai berikut :

$$P_{EC} = \overline{AB}\left(\overline{CD} + CD\right) + \overline{A}B\left(\overline{C}D + C\overline{D}\right) + AB\left(\overline{CD} + CD\right) + A\overline{B}(\overline{C}D + C\overline{D})$$

$$= (\overline{AB} + AB)\left(\overline{CD} + CD\right) + (\overline{A}B + A\overline{B})(\overline{C}D + C\overline{D})$$

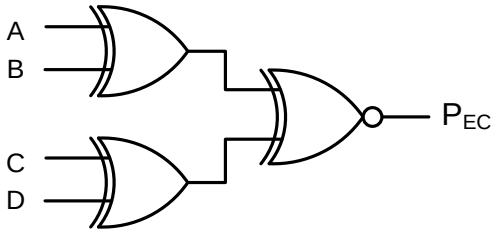
$$= (\overline{A \oplus B})\left(\overline{C \oplus D}\right) + (A \oplus B)(C \oplus D)$$

$$= \overline{XY} + XY$$

$$= \overline{X \oplus Y}$$

$$= \overline{(A \oplus B) \oplus (C \oplus D)}$$

- Rangkaian



Gambar 5. 3 Rangkaian PEC

2. Even Parity

→ Jumlah angka 1 yang dikirimkan berjumlah ganjil.

Contoh Soal :

1) Buatlah rangkaian Parity Generator untuk 3 bit informasi dengan jenis "Even Parity".

Penyelesaian :

- Tabel Kebenaran

Data			Parity
A	B	C	D
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

- Karnaugh-Map

Membuat Persamaan paling sederhana dari tabel kebenaran tersebut menggunakan K-Map.

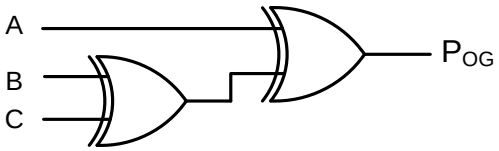
A \ BC	00	01	11	10
0	0	1	0	1
1	1	0	1	0

- Persamaan Output

Berdasarkan K-Map tersebut, persamaan yang dihasilkan adalah sebagai berikut,

$$P_{OG} = \overline{A}\left(\overline{B}C + B\overline{C}\right) + A(\overline{BC} + BC)$$
$$= \overline{A}(B \oplus C) + A(\overline{B \oplus C})$$
$$= \overline{A}X + A\overline{X}$$
$$= A \oplus X$$
$$= A \oplus (B \oplus C)$$

- Rangkaian



Gambar 5. 4 Rangkaian PoG

2) Buatlah Rangkaian Parity Checker untuk 3 bit informasi dengan jenis "Odd Parity".

Penyelesaian :

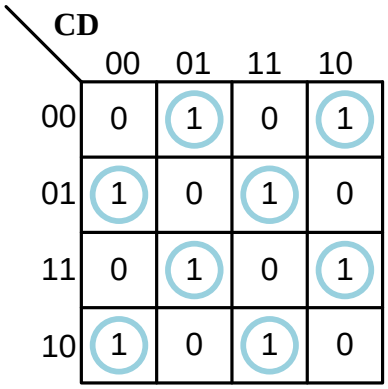
- Tabel Kebenaran

Data			Parity	Parity Error Checker
A	B	C	D	P_{EC}
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0

0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

- Karnaugh-Map

Membuat Persamaan paling sederhana dari tabel kebenaran tersebut menggunakan K-Map.



- Persamaan Output

Berdasarkan K-Map tersebut, persamaan yang dihasilkan adalah sebagai berikut,

$$PEC = \overline{A}\overline{B}(\overline{C}D + C\overline{D}) + \overline{A}B(\overline{C}\overline{D} + CD) + AB(\overline{C}D + C\overline{D}) + A\overline{B}(\overline{C}\overline{D} + CD)$$

$$= (\bar{A}\bar{B} + AB)(\bar{C}D + C\bar{D}) + (\bar{A}B + A\bar{B})(\bar{C}\bar{D} + CD)$$

$$= \overline{(A \oplus B)}(C \oplus D) + (A \oplus B)\overline{(C \oplus D)}$$

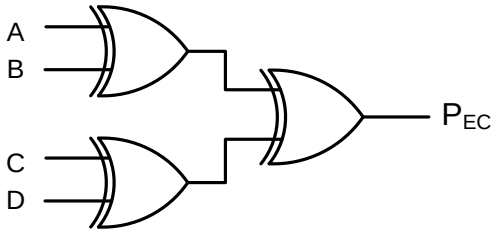
Dimisalkan, A⊕B = X dan C⊕D = Y

$= \bar{X}Y + X\bar{Y}$

$= X \oplus Y$

$= (A \oplus B) \oplus (C \oplus D)$

- Rangkaian



Gambar 5. 5 Rangkaian PEC

BAB 6
PENGGUNAAN PROTEUS

6.1 Pegertian dan Fungsi Proteus

Proteus adalah sebuah software untuk mendesain PCB yang juga dilengkapi dengan simulasi pspice pada level skematik sebelum rangkaian skematik diupgrade ke PCB shingga sebelum PCB nya di cetak kita akan tahu apakah PCB yang akan kita cetak sudah benar atau tidak. Proteus mengkombinasikan program ISIS untuk membuat skematik desain rangkaian dengan program ARES untuk membuat layout PCB dari skematik yang kita buat. Software ini bagus digunakan untuk desain rangkaian mikrokontroller. Proteus juga bagus untuk belajar elektronika seperti dasar-dasar elektronika sampai pada aplikasi mikrokontroller. Software ini jika di install menyediakan banyak contoh aplikasi desain yang disertakan sehingga kita bisa belajar dari contoh-contoh yang sudah ada.

Fitur-fitur dari PROTEUS adalah sebagai berikut :

1. Memiliki kemampuan untuk mensimulasikan hasil rancangan baik digital maupun analog maupun gabungan keduanya.
2. Mendukung simulasi yang menarik dan simulasi secara grafis.
3. Mendukung simulasi berbagai jenis microcontroller seperti PIC, 8051 series.
4. Memiliki model-model peripheral yang interactive seperti LED, tampilan LCD, RS232, dan berbagai jenis library lainnya.
5. Mendukung instrument-instrument virtual seperti voltmeter, ammeter, oscciloscope, logic analyser, dll.
6. Memiliki kemampuan menampilkan berbagi jenis analisis secara grafis seperti transient, frekuensi, noise, distorsi, AC dan DC, dll.
7. Mendukung berbagai jenis komponen-komponen analog.
8. Mendukung open architecture sehingga kita bisa memasukkan program seperti C++ untuk keperluan simulasi.

6.2 Cara Menginstal Proteus pada Window

Cara menginstal proteus sangat mudah, yaitu sebagai berikut :

1. Download aplikasi proteus 8 dari alat dan bahan. Simpan didalam folder komputer anda.
2. Ekstrak Proteus 8 menggunakan aplikasi 7zip. Caranya klik kanan|Klik 7zip|Klik extract to...
3. Buka hasil ekstraksi dan klik 2x pada file setup.exe, jika ada peringatan, klik YES.
4. Jendela proses install Proteus 8 akan dimulai. Lalu klik Next.
5. Ceklist I Accept the terms of this agrement. Lalu klik Next.

6.3 Langkah Membuat Rangkaian pada Proteus

Terdapat beberapa langkah membuat rangkaian pada proteus, yaitu sebagai berikut :

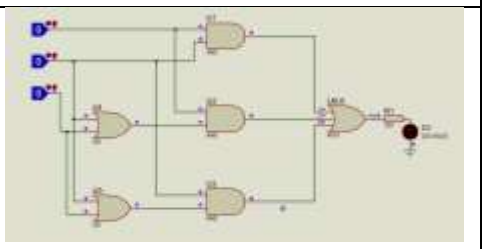
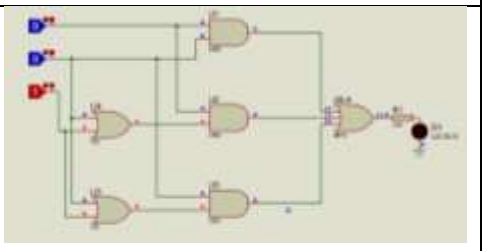
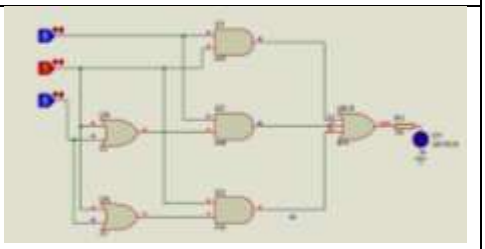
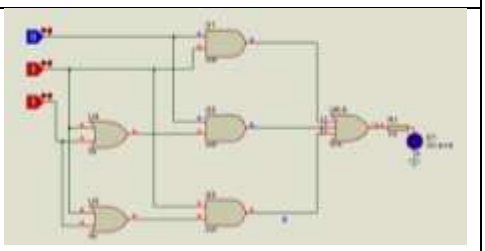
1. Membuat file baru pada proteus.
2. Menambahkan gate dan komponen lain pada library.
3. Merangkai rangkaian pada aplikasi proteus.
4. Setelah rangkaian selesai, pastikan rangkaian dengan cara menekan tombol “run”.
5. Ubah bilangan logic 0 dan 1 ketika simulasi sedang dimulai.

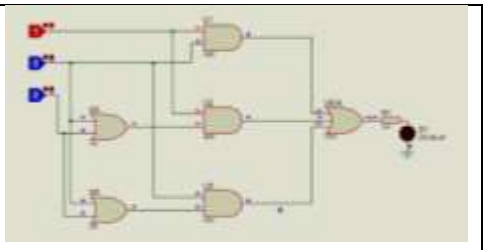
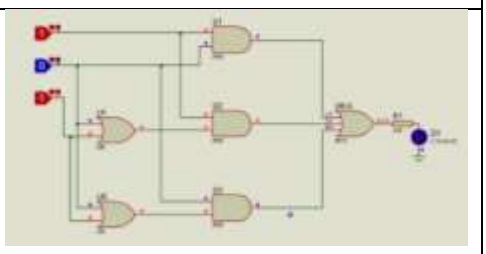
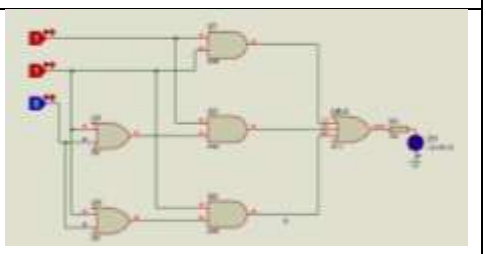
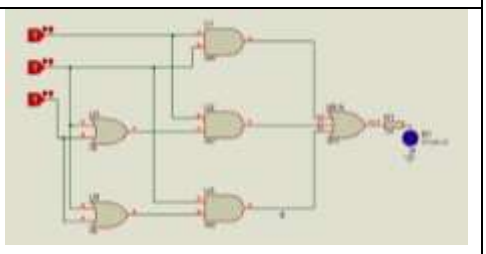
6.4 Contoh dan Latihan Soal

1. Penyederhanaan persamaan menggunakan Aljabar Boolean dan Teorema De Morgan

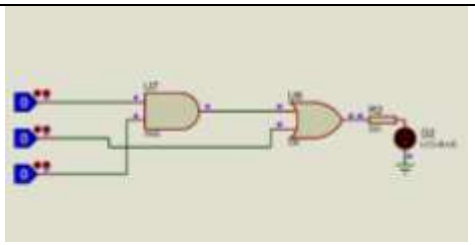
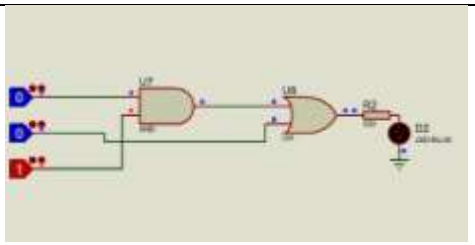
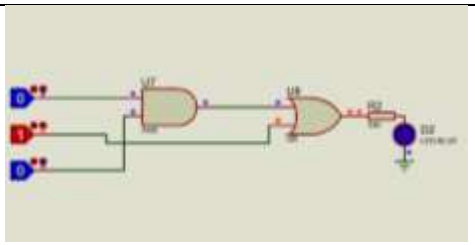
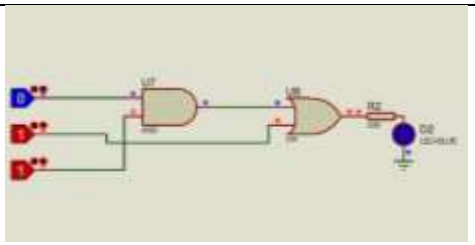
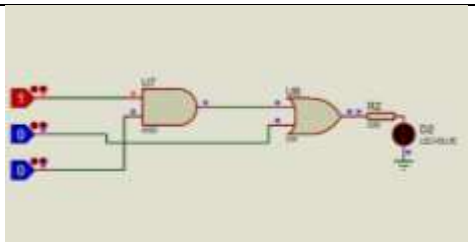
   Tabel dan foto praktikum persamaan $[\mathrm{F} = \mathrm{AB} + \mathrm{A}(\mathrm{B} + \mathrm{C}) + \mathrm{B}(\mathrm{B} + \mathrm{C}) = \mathrm{B} + \mathrm{AC}]$, yaitu sebagai berikut :

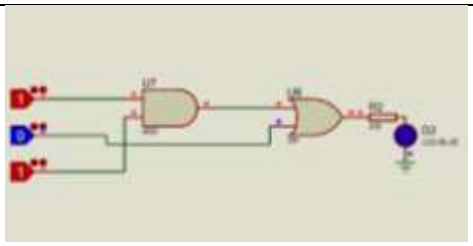
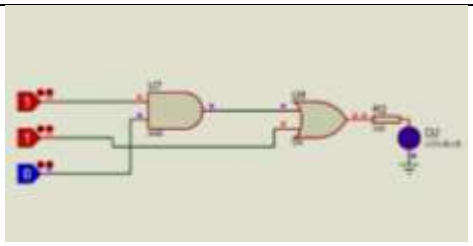
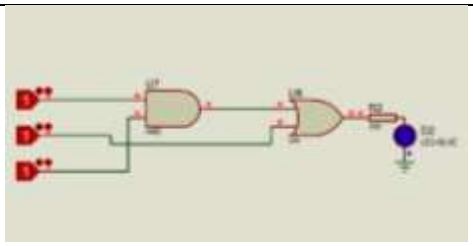
   Ruas Kiri $[\mathrm{F} = \mathrm{AB} + \mathrm{A}(\mathrm{B} + \mathrm{C}) + \mathrm{B}(\mathrm{B} + \mathrm{C})]$

Input			Output	Rangkaian
A	B	C	$AB + A(B + C) + B(B + C)$	
0	0	0	0	
0	0	1	0	
0	1	0	1	
0	1	1	1	

1	0	0	0	
1	0	1	1	
1	1	0	1	
1	1	1	1	

Ruas Kanan [F = B + AC]

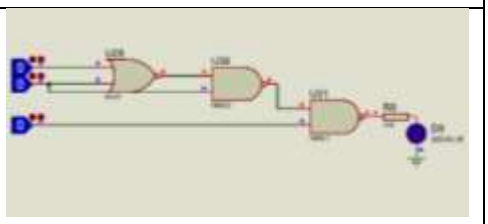
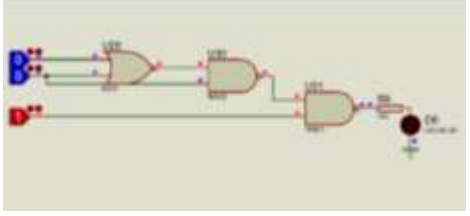
Input			Output	Rangkaian
A	B	C	$AC + B$	
0	0	0	0	
0	0	1	0	
0	1	0	1	
0	1	1	1	
1	0	0	0	

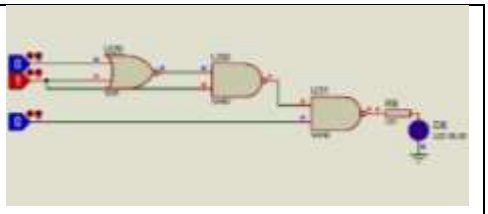
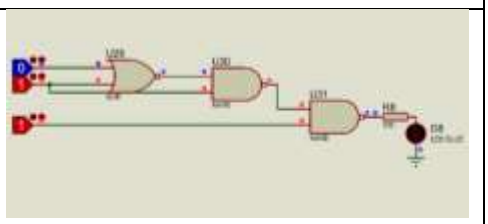
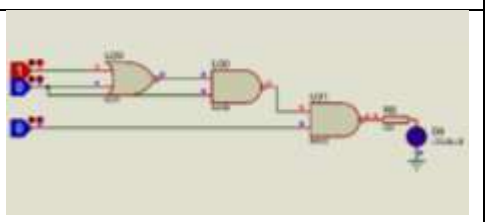
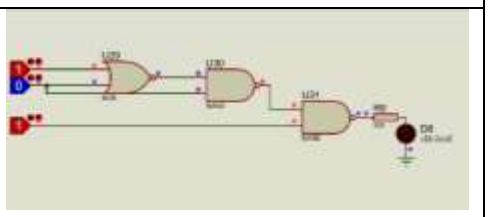
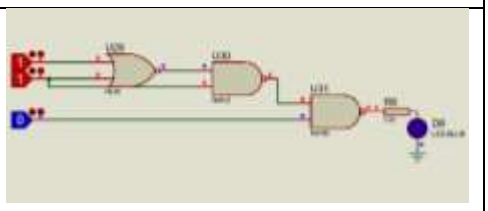
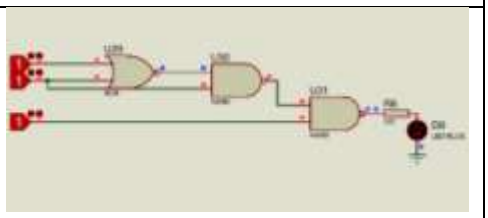
1	0	1	1	
1	1	0	1	
1	1	1	1	

2. Implementasi Odd-Even Method pada suatu rangkaian

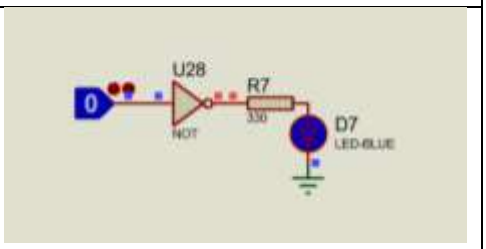
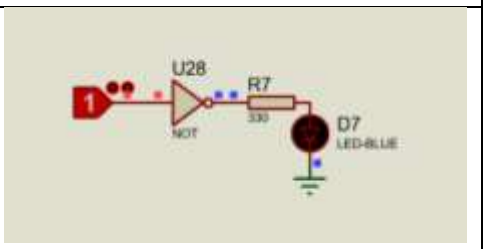
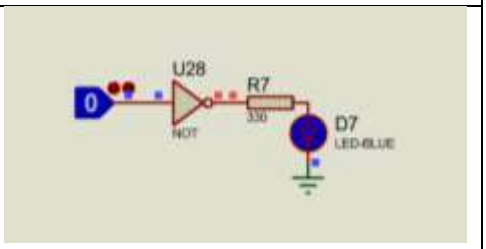
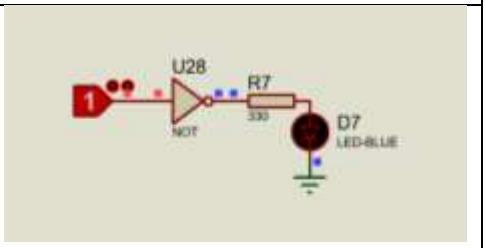
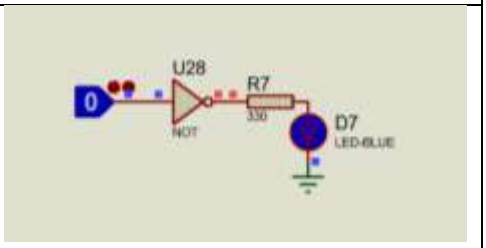
Tabel dan foto praktikum pada persamaan [F = $\bar{A}\bar{B}B + \bar{C} = \bar{C}$], yaitu sebagai berikut :

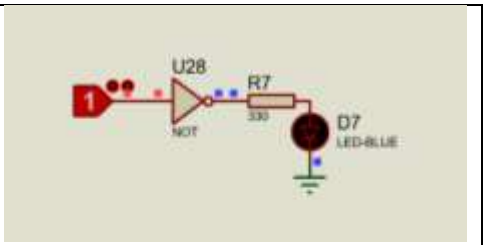
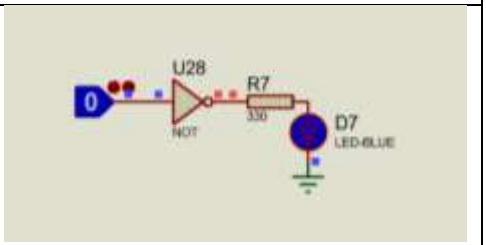
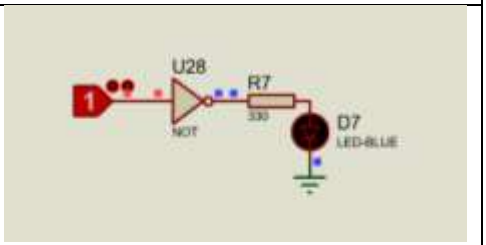
Ruas Kiri [F = $\bar{A}\bar{B}B + \bar{C}$]

Input			Output	Rangkaian
A	B	C	$\bar{A}\bar{B}B + \bar{C}$	
0	0	0	1	
0	0	1	0	

0	1	0	1	
0	1	1	0	
1	0	0	1	
1	0	1	0	
1	1	0	1	
1	1	1	0	

Ruas Kanan [$F = \bar{C}$]

Input			Output	Rangkaian
A	B	C	$\bar{C}$	
0	0	0	1	
0	0	1	0	
0	1	0	1	
0	1	1	0	
1	0	0	1	

1	0	1	0	
1	1	0	1	
1	1	1	0	

DAFTAR PUSTAKA

Hari. 2015. Karnaugh Map atau Peta Karaugh. Riau, Indonesia: Universitas Islam Kuantan Singingi. *Karnaugh Map atau Peta Karnaugh - Harianja Uniks (unikshariania.com)*

Widianto, Mochammad Haldi. 2019. Metode K-Maps. Bandung, Indonesia: BINUS University. *Metode K-Maps | BINUS UNIVERSITY BANDUNG - Kampus Teknologi Kreatif*

Widyanto, Angga Chrisna. 2021. Pengertian Half Adder dan Full Adder. Klaten, Indonesia: SMK Muhammadiyah 2 Klaten Utara. *PENGERTIAN HALF ADDER DAN FULL ADDER (smkmuh2klaten.sch.id)*

Pulkit, dkk. 2022. Difference Between Half Adder and Full Adder. Uttar Pradesh, India: Geeks for Geeks. *Difference between Half adder and full adder - GeeksforGeeks*

Satiawan, Hendra. 2014. *Sistem Bilangan Digital*. Sistem Bilangan Digital - ppt download (slideplayer.info)

Sugiartowo, Sitti Nurbaya Ambo. 2018. *Simulasi Rangkaian Kombinasional sebagai Media Pembelajaran Sistem Digital pada Fakultas Teknik Universitas Muhammadiyah Jakarta*. Jakarta Pusat, Indonesia: Program Studi Tenik Informatika Universitas Muhammadiyah Jakarta. 3407-8225-1-PB.pdf

Nurmiati, Evy.2015. *Sistemm Bilangan: Pengantar Teknologi Informasi*. Pamulang, Indonesia: Program Studi Teknik Informasika Universitas Pamulang. SISTEM BILANGAN Pengantar Teknologi Informasi - ppt download (slideplayer.info)

Susanti, Rika. 2014. *Sistem Bilangan dan Kode Bilangan*. SISTEM BILANGAN DAN KODE BILANGAN - ppt download (slideplayer.info)

Tambunan, Iskandar. 2015. *Rangkaian Adder*. Rangkaian Adder (slideshare.net)